University of Stuttgart
Germany

Institute of
Robust Power
Semiconductor Systems

Automated Sensor Fusion for Failure Detection in Power Semiconductor

Master's Thesis

Submitted by

Chaitanya Manojkumar Mhetre

from Stuttgart, Germany

at the Institute of Robust Power Semiconductor Systems

of the University of Stuttgart

Supervisor: M.Sc. Valentyna Afanasenko
Prof. Dr.-Ing. Ingmar Kallfass

Period: 15.10.2024 – 15.04.2025

Stuttgart, 15.04.2025

Postanschrift: Institut für Robuste Leistungshalbleitersysteme
Pfaffenwaldring 47

D-70569 Stuttgart

Tel.: +49 (0) 711 685 68700
Fax.: +49 (0) 711 685 58700
E-Mail: sekretariat@ilh.uni-stuttgart.de
Web: www.ilh.uni-stuttgart.de

Overview

Type of thesis:	Master's Thesis
English title:	Automated Sensor Fusion for Failure Detection in Power Semiconductor
German title:	Automatisierte Sensorfusion zur Fehlererkennung in Leistungshalbleiten
Student:	Chaitanya Manojkumar Mhetre
Supervisor:	M.Sc. Valentyna Afanasenko
Date of Examination:	15.04.2025

Statutory Declaration

I hereby declare that this thesis is my own work and effort and follows the regulations related to good scientific practice of the University of Stuttgart in its latest form. All sources cited or quoted are indicated and acknowledged by means of a comprehensive list of references.

Stuttgart, 15.04.2025

Chaitanya Manojkumar Mhetre

Executive Abstract

The complexity, heterogeneity and multimodal nature of the data involved make failure analysis of power semiconductor devices extremely difficult. Traditional failure analysis methods frequently depend on the utilization of single-modal data, which can result in suboptimal detection accuracy and required extensive manual pre-processing. In order to improve failure classification in Gallium Nitride (GaN) High Electron Mobility Transistors (HEMT), this thesis explores the application of automated sensor fusion approaches, through integration of time-domain reflection (TDR) signals and S-parameter data.

The aim of this thesis is to create an AI-based approach for failure classification in Transistor. In addition, to implement a method to automate data synchronization, normalization, and fusion procedures. By using a Transformer model for failure classification, the algorithm improves the accuracy for failure classification when examined with Uni-modal (TDR data) and Multi-modal (synchronized s-parameter and TDR data). A functionality is created with input of raw s-parameter and TDR data files, creates a dataset which can be directly used for the classification task. This functionality creates a unified multimodal data, and it is normalized for the transformer model.

Furthermore, the manual efforts in data preprocessing is reduced by implementing automated sensor data fusion. The implemented Transformer model shows an improvement of 6.67% with synchronized data over TDR data and the model accuracy improves by 11.67% with the use of transfer learning.

Zusammenfassung

Die Komplexität, Heterogenität und Multimodalität der Daten machen die Fehleranalyse von Leistungshalbleitern extrem schwierig. Herkömmliche Fehleranalysemethoden hängen häufig von der Verwendung einmodaler Daten ab, was zu einer suboptimalen Erkennungsgenauigkeit führen kann und eine umfangreiche manuelle Vorverarbeitung erfordert. Um die Fehlerklassifizierung in Galliumnitrid (GaN) High Electron Mobility Transistors (HEMT) zu verbessern, wird in dieser Arbeit die Anwendung automatisierter Sensorfusionsansätze durch Integration von Zeitbereichsreflexionssignalen (TDR) und S-Parameterdaten untersucht.

Ziel dieser Arbeit ist es, einen KI-basierten Ansatz zur Fehlerklassifizierung in Transistoren zu entwickeln. Darüber hinaus soll eine Methode zur Automatisierung der Datensynchronisierung, Normalisierung und Fusionierung implementiert werden. Durch die Verwendung eines Transformer-Modells zur Fehlerklassifizierung verbessert der Algorithmus die Genauigkeit der Fehlerklassifizierung bei der Untersuchung von unimodalen (TDR-Daten) und multimodalen (synchronisierte s-Parameter und TDR-Daten) Daten. Mit der Eingabe von rohen s-Parameter- und TDR-Datendateien wird eine Funktionalität geschaffen, die einen Datensatz erzeugt, der direkt für die Klassifizierungsaufgabe verwendet werden kann. Diese Funktion erzeugt einheitliche multimodale Daten, die für das Transformatormodell normalisiert werden.

Darüber hinaus wird der manuelle Aufwand bei der Datenvorverarbeitung durch die Implementierung einer automatisierten Sensordatenfusion reduziert. Das implementierte Transformer-Modell zeigt mit synchronisierten Daten eine Verbesserung von 6.67% gegenüber TDR-Daten, und die Modellgenauigkeit verbessert sich durch den Einsatz von Transferlernen um 11.67%.

Contents

Overview **i**

Statutory Declaration **iii**

Executive Abstract **v**

Zusammenfassung **vii**

Abbreviations and Symbols **xv**

Abbreviations xv

1 Introduction **1**

2 Fundamentals **3**

2.1 GaN HEMT Transistor 3

2.2 Failure detection in power semiconductor devices 5

2.3 Multi-modal data 5

2.4 Multi-modal Classification 6

2.5 Data Fusion Technique 7

2.5.1 Early Fusion 8

2.5.2 Intermediate Fusion 8

2.5.3 Late Fusion 9

2.6 Transfer Learning 9

2.7 Modality imputation in Multi-modal learning 11

2.8 Data Pre-processing and Feature Selection in Multi-modal Learning 12

2.9 Short introduction of Transformer Model 14

2.9.1 Architecture of the Transformer Model 14

2.9.2 Multi-Modal Transformer 18

2.9.3 Encoder-Only Transformer 19

3 State of The Art **23**

3.1 Available Models for Multi-modal Classification 23

3.2 Selection of Suitable model for Multi-modal classification 25
3.2.1 Sequential data 25
3.2.2 Challenges of Sequential Data in Classification 26
3.2.3 Challenges for Multi-modal Classification 27
3.2.4 Comparison of different Models for the Multi-modal classification 27

4 Implementation 29
4.1 Dataset Description 29
4.1.1 s11 and TDR plots for DUT 1 and 5 29
4.1.2 S-parameter and TDR Dataset 30
4.2 Data pre-processing and Synchronization 31
4.3 Data Scaling and splitting 32
4.3.1 Dataset Synchronization 32
4.3.2 Data Scaling and Tensor Conversion 33
4.3.3 Dataset Preparation for Model Training 33
4.4 Setup for Model Training 33
4.5 Transformer Model for classification 35
4.5.1 Uni-Modal Transformer Model 35
4.5.2 Multi-modal Transformer Model 36
4.5.3 Multi-modal Transformer Model with Data imputation 37
4.5.4 Trained Multi-modal Transformer Model 38

5 Results 39
5.1 Uni-modal transformer performance evaluation 39
5.2 Multi-modal transformer performance evaluation 41
5.3 Comparative analysis 43

6 Conclusion and Outlook 45

Bibliography 47

List of Figures

2.1 Multi-modal Classification 7
2.2 Overview of different Data Fusion Techniques 8
2.3 Overview of Transfer Learning 10
2.4 Block diagram of Modality imputation with zero values 12
2.5 Transformer Architecture 15
2.6 Multi-Head Attention Mechanism 16
2.7 A typical feed-forward network 17
2.8 An example of Positional Encoding 17
2.9 Working mechanism of Positional Encoding with word embedding 18
2.10 Structure of Multi-modal Transformer 19

4.1 s11 and Time-Domain Reflectometry (TDR) plots for device under test (DUT) 1 and 5 30
4.2 Single Modality Transformer Architecture 35
4.3 Multi-Modal Transformer Architecture 37

5.1 Test accuracy results for Uni-modal transformer 40
5.2 Confusion matrix for Uni-modal transformer(TDR 30 samples) 40
5.3 Test accuracy results for Multi-modal transformer 42
5.4 Confusion matrix for Multi-modal transformer model 43
5.5 Comparative results of Transformer models 44

List of Tables

2.1 Key Parameters of gallium nitride (GaN) high-electron-mobility transistor (HEMT) Transistor . . . 3
2.2 Comparison of Different Scaling Techniques . . . 13
2.3 Usage of different Scaling Technique . . . 14

3.1 Comparison of models for Multi-modal classification . . . 28

4.1 DUT value definition . . . 30
4.2 Values of key parameters . . . 31
4.3 Values of key parameters . . . 32
4.4 Training Parameters . . . 34

5.1 Dataset configuration for model training and testing . . . 39

Abbreviations and Symbols

Abbreviations

Notation	Description
2DEG	two-dimensional electron gas
AI	Artificial Intelligence
AlGaN	Aluminum Gallium Nitride
BERT	Bidirectional Encoder Representations from Transformers
CNNs	convolutional neural networks
DNNs	Deep Neural Networks
DUT	device under test
FFNs	Feed-Forward Networks
GaN	gallium nitride
GANs	generative adversarial networks
GRU	Gated Recurrent Unit
HEMT	high-electron-mobility transistor
KNNs	k-nearest neighbor
LSTM	Long short-term memory
MDAEs	multi-modal deep autoencoders
ML	Machine Learning
MulT	Multi-modal Transformers
NER	named entity recognition
NLP	Natural language processing
ReLU	rectified linear unit
RF	radio frequency
RFE	recursive feature elimination
RNNs	recurrent neural networks

Notation	Description
SVM	support vector machines
TDR	Time-Domain Reflectometry
VAEs	Variational auto-encoders
ViLT	Vision-Language Transformers
VNA	vector network analyzer

1 Introduction

A GaN HEMT is a type of transistor that are widely used in high-frequency, high-power applications such as RF amplifiers, power electronics, and microwave communications. Their ability to handle high voltages and operate efficiently at high speeds makes them superior to traditional silicon-based transistors in many advanced applications.

Low-frequency S-parameter measurements have been utilized to highlight trapping phenomena in GaN HEMT. These measurements can reveal changes in impedance and gain compression, indicative of electron trapping effects [49]. The degradation of the gate Schottky junction, often induced by off-state stress, leads to an increase in gate leakage current. This degradation is an effect of shifts in the input reflection coefficient (S11), signaling gate degradation [17]. TDR techniques are effective in analyzing impedance variations along the transistor structure, aiding in the localization of failure sites such as gate-drain breakdown and channel defects. These measurements provide insights into physical defects or material degradation within the device [23]. This example shows that S-parameter and TDR data give an insight into the cause of failures in GaN HEMT transistor.

Combining multiple data modalities such as electrical, optical, thermal, and acoustic signals for failure detection or parameter estimation in semiconductor devices, has emerged as a promising approach. This enhances accuracy and robustness over single-modality methods. Recent research demonstrates that multimodal data fusion enables the early detection of subtle anomalies that may not be evident in isolated data streams. Integrating thermal imaging with electrical measurements improves fault localization in power semiconductor modules, especially under dynamic operating conditions [25]. Similarly, combining time-resolved photoluminescence and I-V curve analysis yields more precise estimations of degradation parameters in photovoltaic cells [43]. The primary advantages include improved reliability, reduced false positives, and greater interpretability in complex device behavior. However, key challenges persist, such as aligning heterogeneous data sources in time and space, dealing with varying data quality, and the increased computational cost of processing and modeling such high-dimensional inputs.

Recent advancements in Artificial Intelligence (AI) and Machine Learning (ML) have significantly enhanced the ability to process multi-modal data, perform sensor fusion [4]. These approaches integrate multiple data to improve diagnostic accuracy and reduce reliance on manual pre-processing. By implementing Transformer algorithms and their self-attention mechanisms, the model is capa-

ble of handling multi-modal data and leads to better accuracy for failure classification [47].

The failure detection in GaN HEMT is complex due to the diverse types of data involved. For uni-modal models, the difficulty is that using only one type of data may not fully capture the complex failure mechanisms. In the context of multi-modal models, the challenge arises from combining multiple data sources with different characteristics through sensor fusion. Conventional methods rely on features extracted from electrical and thermal signals, limiting their scalability and adaptability. Moreover, existing ML models often operate on single-modality data, which restricts their ability to capture interactions across different data types. To address these challenges, this study focuses on the development of a multi-modal classification model that integrates S-parameters and TDR data for improved failure detection. Also, to develop a framework that reduces human intervention in feature extraction and pre-processing while maintaining high detection accuracy.

The main objective of this thesis is to select a suitable ML model of failure classification of transistors and to create an automated data fusion technique. Chapter 2 explains all the fundamental concepts related to the task. This chapter provides an in-depth review of the existing literature on failure classification in transistors, data preparation, different sensor fusion methodologies and Transfer learning. Chapter 3 outlines different available algorithms for multi-modal classification, the adaptability of these algorithms for the given data and a comparative overview of them. Chapter 4 presents the complete workflow of the task, implementation of the selected model, model training process and the function of the automated sensor fusion technique. Chapter 5 discusses the results, compares them with different approaches of Uni-modal and Multi-modal Transformer model and analyzes their implications. Chapter 6 concludes the thesis with key findings, contributions, and recommendations for future research.

2 Fundamentals

In this chapter, the theoretical foundations of GaN HEMT transistor, failure analysis in transistor, multi-modal data, multi-modal classification, data fusion technique, modality imputation in multi-modal learning, concept of transfer learning, data pre-processing techniques and transformer model architecture are described and the important aspects to be considered in the design are given in detail.

2.1 GaN HEMT Transistor

GaN HEMT transistors are widely used in high-frequency and high-power applications due to their material properties, including a wide bandgap, high electron mobility, and excellent thermal stability. These characteristics enable GaN HEMT to operate at higher voltages, frequencies, and efficiencies compared to traditional silicon-based transistors, making them ideal for radio frequency (RF) and power electronics applications. Additionally, the presence of a two-dimensional electron gas (2DEG) at the Aluminum Gallium Nitride (AlGaN)/ GaN hetero-junction enhances carrier transport, reducing on-resistance and improving overall device performance. Recent advancements in GaN HEMT technology have focused on enhancing reliability, minimizing trapping effects, and optimizing thermal management to support next-generation communication and power conversion systems. [27]

For GaN HEMT transistor, the parameters V_{ds}, V_{gs} and L_{g} are considered for this thesis work and their description is given in Table 2.1:

Table 2.1: Key Parameters of GaN HEMT Transistor

Parameter	Symbol	Description
Drain-Source Voltage	V_{ds}	Voltage between drain and source terminals
Gate-Source Voltage	V_{gs}	Voltage between gate and source terminals
Gate Length	L_{g}	Length of the gate electrode

Drain-Source Voltage (V_{ds}):

This is the voltage applied between the drain and source terminals of the transistor. GaN HEMT can operate at higher V_{ds} compared to silicon-based transistors due to GaN's wide bandgap, typically in the range of 10V to 600V or more, depending on the application (RF vs. power

electronics). A higher V_{ds} enables increased power density and efficiency, making GaN HEMT ideal for power conversion and RF amplification.

Gate-Source Voltage (V_{gs}):

This is the voltage applied between the gate and source terminals, which controls the channel conductivity. GaN HEMT transistors are usually normally on (depletion mode) but can be engineered to be normally off (enhancement mode) for safer operation. The typical threshold voltage V_{gs} for enhancement-mode GaN HEMT transistor is around 0V to $+$2V, with a maximum V_{gs} limit usually between -10V to $+$6V to prevent gate damage.

Gate Length (L_{g}):

This represents the physical length of the gate electrode, which directly impacts the transistor's frequency response and short-channel effects. For RF and high-speed applications, L_{g} is typically below 150 nm, sometimes reaching sub-100 nm to maximize high-frequency operation. For power applications, L_{g} is relatively longer (e.g., 0.5–2 µm) to handle higher voltages and reduce leakage currents.

For GaN HEMT transistor, 2 types of data will be considered for this work. A brief information about this is as follows:

- **S-Parameters:** S-parameters (scattering parameters) characterize the small-signal behavior of GaN HEMT devices in the frequency domain. They describe how RF signals are transmitted and reflected at different ports, providing crucial information about gain, input/output impedance, and stability. Due to the high power density and wide bandwidth capabilities of GaN HEMT transistors, their S-parameters are essential for designing RF and microwave circuits, including power amplifiers and low-noise amplifiers. These parameters are typically measured using a vector network analyzer (VNA) over a range of frequencies to optimize matching networks and ensure efficient power transfer.
- **Time-Domain Reflectometry (TDR):** TDR is a technique used to analyze the impedance characteristics and signal integrity of GaN HEMT transistors. By sending a fast-rising edge pulse into the transistor and measuring the reflections, TDR provides insight into discontinuities, parasitic effects, and transmission line mismatches. This is particularly useful in high-frequency and high-power applications where precise impedance matching is critical. TDR measurements help in optimizing packaging, reducing signal loss, and ensuring reliable performance of GaN-based RF and power devices.

2.2 Failure detection in power semiconductor devices

Failure detection in power semiconductor devices using ML classification models has emerged as a promising approach to enhance the reliability and operational safety of power electronic systems. By leveraging historical operational data such as voltage, current, temperature, and switching behavior, glsml algorithms can learn patterns associated with normal and faulty conditions. Supervised classification models such as support vector machines (SVM), random forests, and deep neural networks are commonly employed to distinguish between healthy and degraded states of power semiconductordevices. For example, glsml algorithms have been used to predict device reliability under extreme conditions, demonstrating the potential of data-driven models in real-world scenarios [30]. A binary classification model was also developed for arc fault detection in electric vehicle battery systems, achieving high accuracy through feature learning [46]. These approaches significantly enhance early fault detection, often outperforming traditional threshold-based methods by identifying subtle anomalies. The integration of ML based classification into predictive maintenance strategies can reduce system downtime, improve fault prognosis, and extend the lifespan of power semiconductor components [33].

For GaN HEMT transistors, S-parameters provide information about input reflection and impedance matching, which are vital for evaluating the integrity of the transistor's input network. Meanwhile, TDR data helps identify impedance changes along the transmission path, revealing defects or irregularities within the device. By combining insights from both s11 and TDR data, it can be used to accurately locate failure sites, such as voids or de-laminations, and understand their effects on performance [51]. This comprehensive analysis improves the accuracy of failure diagnosis and supports better reliability strategies for power semiconductor devices.

2.3 Multi-modal data

Multi-modal data refers to information collected from multiple sources or sensors that operate across different modalities, such as textual, visual, auditory, and physiological data. The integration and analysis of multi-modal data are critical in various domains, including AI, ML, healthcare, robotics, and multimedia retrieval. The primary advantage of multi-modal data is that it enables a more comprehensive and holistic understanding of a given phenomenon by leveraging the strengths of each modality.

Multi-modal data exhibits several key characteristics. Firstly, it is inherently heterogeneous, encompassing diverse data types such as structured numerical data, images, text, and speech. Secondly, it provides complementarity, where different modalities contribute unique and often non-overlapping perspectives, leading to a richer understanding. Additionally, temporal alignment

is a crucial aspect in multi-modal data, as different data streams may have varying timestamps and resolutions that require synchronization. The integration of multi-modal data often involves fusion strategies, such as early fusion, where raw data from different modalities are combined at the input level, and late fusion, where extracted features from each modality are merged at a later stage [4].

The applications of multi-modal data are vast and span multiple industries. In healthcare, the combination of medical imaging, patient history, and physiological signals improves disease diagnosis and treatment planning. Autonomous vehicles rely on sensor fusion from different sources to robust navigation and accurate object detection. In Natural language processing (NLP), advanced models can enhance understanding by integrating text and images, facilitating more sophisticated AI interactions. Additionally, human-computer interaction benefits from multi-modal data through the combination of speech, facial expressions, and gestures, enabling more intuitive and adaptive user experiences [29].

Despite its advantages, multi-modal data processing presents several challenges. Data alignment is a major issue, as different sources often have varying timestamps and resolutions, making synchronization complex. Noise and redundancy also pose significant hurdles, as some modalities may contain irrelevant or duplicate information that must be filtered out. Moreover, handling large-scale, high-dimensional, multi-modal data demands substantial computational resources, necessitating efficient algorithms and optimized hardware.

2.4 Multi-modal Classification

Multi-modal classification is a machine learning approach that involves integrating data from multiple modalities, such as text, images, audio, and video, to improve classification performance. Using information from different modalities it leads to richer feature representations and more robust predictions. The growing availability of diverse data sources has fueled the development of multi-modal learning, with applications in healthcare, autonomous driving, and human-computer interaction. Recent advances, particularly in deep learning and transformer-based architectures, have significantly improved the ability to process and fuse multi-modal data effectively [40].

Despite its advantages, multi-modal classification presents several challenges. One key issue is representation learning, where models must capture meaningful features from heterogeneous data types that differ in structure and semantics. Additionally, aligning and integrating information across modalities is complex due to differences in spatial and temporal characteristics. Methods such as early fusion (combining raw data before feature extraction), late fusion (combining outputs of uni-modal classifiers), and hybrid fusion aim to address these challenges [4]. Moreover, co-learning strategies, which leverage relationships between modalities to improve learning even when

some modalities are missing, have emerged as a promising direction [16].

Advancements in multi-modal classification have been largely driven by transformer-based architectures and self-supervised learning techniques. Models like Multi-modal Transformers (MulT) and Vision-Language Transformers (ViLT) have demonstrated strong performance by effectively capturing cross-modal interactions [16]. Furthermore, multi-modal classification has various fusion approaches, including attention mechanisms and graph-based approaches, to enhance model interpretability and robustness [22]. As multi-modal learning continues to evolve, improving scalability challenges and improving generalization across diverse datasets. Developing more efficient training strategies is possible.

Figure 2.1 shows a functional block diagram of multi-modal classification.

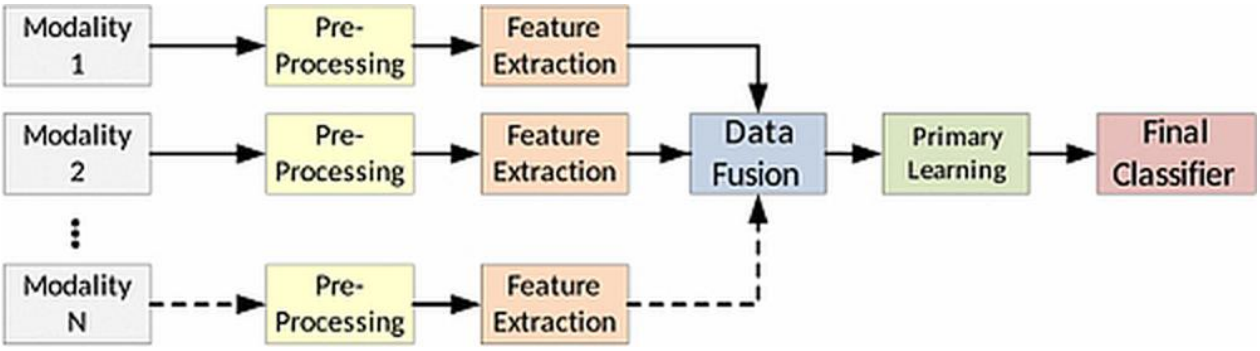


Figure 2.1: Multi-modal Classification
[37]

2.5 Data Fusion Technique

Data fusion is a technique for integrating multiple heterogeneous datasets to derive meaningful features from data. Traditional data mining techniques primarily focus on single-domain data, whereas modern data fusion methods utilize information from diverse sources to enhance decision-making and predictive modeling. This section explores various data fusion methodologies, including stage-based fusion, feature-level fusion, and semantic meaning-based fusion [35].

Data fusion can be broadly classified into three main categories:

- **Early Fusion**
- **Intermediate Fusion**
- **Late Fusion**

The broad classification of different fusion techniques is given in figure 2.2. The sub-figure (a) gives an example of Early fusion with the help of a block diagram. Similarly, sub-figure (b) is an example of Intermediate fusion and sub-figure (c) is an example of the Late fusion technique.

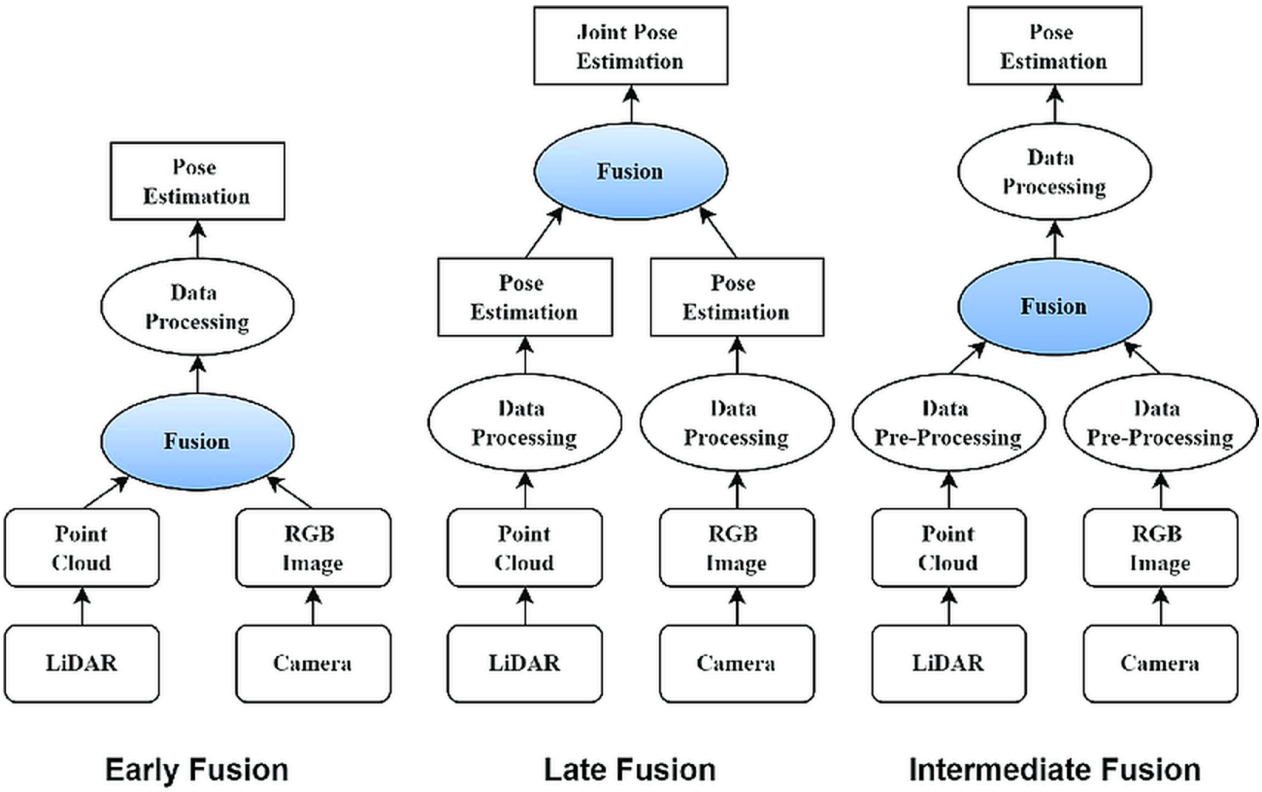


Figure 2.2: Overview of different Data Fusion Techniques
[1]

2.5.1 Early Fusion

Early fusion, also known as feature-level fusion, involves integrating multiple data sources at the initial stages of processing to create a unified representation. In power electronics, this technique has been applied to enhance system monitoring and fault detection. For example, in the context of particle accelerator power electronics, early fusion methods have been employed to combine various sensor data, enabling more accurate and timely detection of system anomalies. By merging diverse data streams early in the processing pipeline, these systems can capture intricate relationships between different operational parameters, leading to improved reliability and efficiency in power electronic applications [32].

2.5.2 Intermediate Fusion

Intermediate (Feature-level) fusion methods aim to integrate multiple datasets by combining their extracted features into a unified representation. This category can be further divided into:

- **Direct Concatenation:** Features from different datasets are concatenated into a single feature vector. While simple, this approach often leads to overfitting and does not consider

inter-feature dependencies.

- **Deep Neural Networks (DNN)-Based Fusion:** Advanced machine learning techniques use Deep Neural Networks (DNNs) to learn new feature representations from multi-modal datasets. These models capture non-linear relationships across modalities and improve performance in tasks like image recognition and speech translation [35].

2.5.3 Late Fusion

Unlike feature-level methods that treat features as numerical values, late (semantic meaning-based) fusion methods focus on the interpretability of data relationships. These methods are further divided into:

- **Multi-View Learning:** Different datasets provide complementary views of an object or event. Techniques like co-training use multiple classifiers trained on different views to reinforce learning.
- **Similarity-Based Fusion:** Methods such as coupled matrix factorization and manifold alignment use correlation structures between datasets to enhance predictions. For example, in recommendation systems, user-item interactions can be improved by incorporating contextual features like geographical data and user preferences.
- **Probabilistic Dependency-Based Fusion:** Techniques such as Bayesian networks and Markov Random Fields model dependencies between features from different datasets to infer relationships and improve decision-making.
- **Transfer Learning-Based Fusion:** Knowledge from a well-labeled source domain is transferred to a sparsely labeled target domain, enabling better predictions despite limited data. This approach is useful in applications like cross-domain recommendation systems, where knowledge from movie preferences can be adapted to travel recommendations.

Data fusion plays a crucial role in multi-modal data analysis by integrating disparate datasets to improve accuracy and knowledge discovery. With different techniques as stage-based, feature-level or semantic meaning-based approaches. Data fusion enables better decision-making across various domains. In the field of machine learning and artificial, data fusion techniques is a crucial step and has added value in the data processing and analysis.

2.6 Transfer Learning

Transfer learning is a machine learning technique where a model developed for one task is reused as a starting point for a different but related task. Transfer learning uses pre-trained models rather

than starting from scratch to increase performance and efficiency, especially in situations where data is scarce. In fields including speech recognition, computer vision, and NLP, this method has drawn a lot of interest. To attain high accuracy, traditional machine learning models need a lot of labeled datasets and processing power. However, obtaining labeled data is costly and time-consuming in many real-world situations. By enabling knowledge transfer between domains, transfer learning solves this problem by lowering the requirement for huge datasets and expediting training [52].

Figure 2.3 gives an overview on working of Transfer learning by comparing it with the traditional machine learning approach.

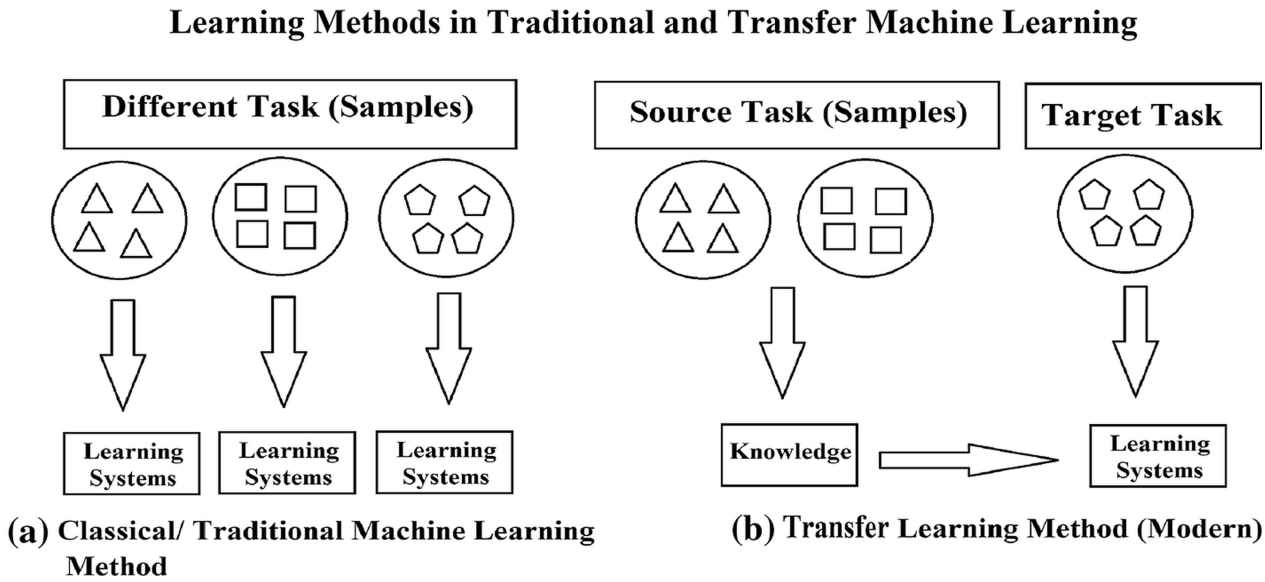


Figure 2.3: Overview of Transfer Learning
[15]

One of the most significant applications of transfer learning is in computer vision. For applications including object detection, facial recognition, and medical imaging, pre-trained convolutional neural networks (CNNs) are frequently utilized. Models have transformed machine translation, sentiment analysis, and text categorization in NLP. Transfer learning's adaptability across industries is further demonstrated by its widespread application in speech recognition, healthcare diagnostics, finance, and business analytics.

The implementation of transfer learning follows a structured workflow. First, a suitable pre-trained model is selected based on its relevance to the target task. Then, feature extraction is performed by freezing the early layers of the model while training only the final layers. Fine-tuning involves unfreezing some of the pre-trained layers and re-training them using the target dataset for better adaptation. Finally, the model is evaluated, optimized, and fine-tuned to ensure high performance. This structured approach enables transfer learning to be applied effectively across

different domains.

Despite its advantages, transfer learning comes with challenges and limitations. Domain shift occurs when the source and target domains differ significantly, leading to poor model generalization. Negative transfer can degrade performance if the knowledge from the source task is not beneficial to the target task. Fine-tuning large pre-trained models also demands significant computational resources. Additionally, biases present in pre-trained models may be transferred to new applications, raising ethical concerns related to fairness and accountability [31].

Transfer learning has evolved with advancements in self-supervised learning, few-shot and zero-shot learning, federated transfer learning and multi-modal learning [45]. Self-supervised learning techniques help models learn meaningful representations without labeled data. Few-shot and zero-shot learning approaches allow models to generalize to new tasks with minimal or no training data. Federated transfer learning enhances data privacy by enabling models to be trained across decentralized datasets. Multi-modal learning integrates information from different sources, such as text and images, to improve various applications [45].

2.7 Modality imputation in Multi-modal learning

Modality imputation is a crucial technique in multi-modal learning, addressing the issue of missing data in scenarios where multiple data modalities are utilized. In many applications, such as healthcare, autonomous driving, and human-computer interaction, data from different modalities—such as text, images, and sensor readings are often incomplete due to sensor failures, occlusions, or data corruption. Imputation methods aim to reconstruct the missing modality, thereby ensuring that the learning model remains robust and effective.

Traditional approaches for handling missing modalities include statistical methods such as mean imputation and k-nearest neighbor (KNNs) imputation. However, these techniques often fail to capture the complex relationships between modalities. A common baseline approach in handling missing modalities is replacing the missing data with zeros. This method, known as zero imputation, is computationally efficient and easy to implement but has significant drawbacks. Zero imputation can introduce biases as it assumes that the absence of a modality does not provide additional information. In many cases, zero replacement leads to degraded model performance since deep learning models may struggle to differentiate between genuinely low-value inputs and missing data.

Figure **??** shows the block diagram for the zero value imputation. Where Modality 2 is missing/ not in proper format, it is replaced with zero values. This helps in keeping the structure of the multi-modal model as same. Even if few samples of Modality 2 are present, they are not

completely ignored due to the missing values.

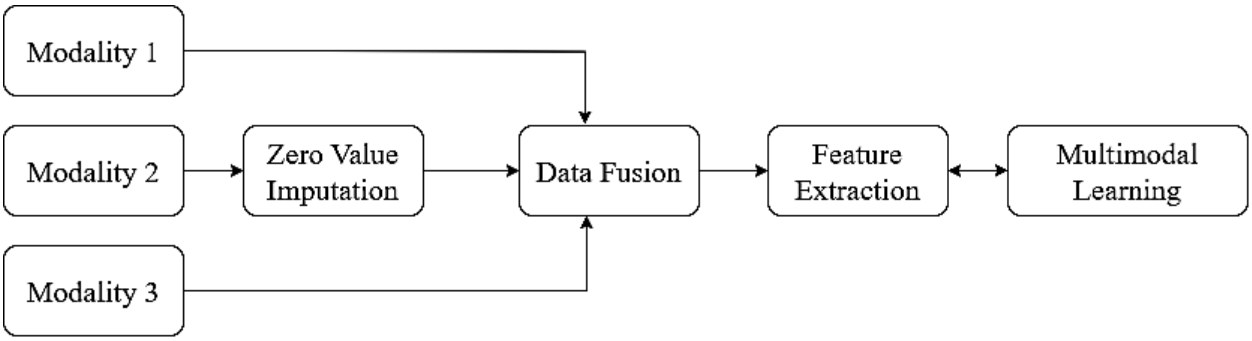


Figure 2.4: Block diagram of Modality imputation with zero values [24]

Recent advancements in deep learning have introduced more sophisticated methods for modality imputation, including generative generative adversarial networks (GANs) and Variational auto-encoders (VAEs), which learn the underlying data distribution to generate plausible missing modality representations [24]. Cross-modal translation models leverage shared representations across modalities to predict missing information. Transformer-based architectures, such as the Multi-modal Transformer [40], have demonstrated remarkable success in learning cross-modal dependencies and imputing missing modalities through attention mechanisms.

Despite the limitations of zero imputation, it remains a widely used approach, particularly in scenarios where computational efficiency is prioritized over accuracy. However, to mitigate its drawbacks, researchers often combine zero imputation with feature-masking techniques or train models to recognize missing data as a special case. Another promising approach is self-supervised learning, where models pre-train on available modalities and later fine-tune to reconstruct missing ones [40].

The main challenge is ensuring the quality and reliability of imputed modalities, particularly when large portions of data are missing. In [40], it focuses on improving model generalization, incorporating domain knowledge and developing hybrid models that integrate multiple imputation techniques. Balancing efficiency and accuracy is the key consideration in choosing the appropriate imputation strategy.

2.8 Data Pre-processing and Feature Selection in Multi-modal Learning

Multi-modal learning involves the integration of multiple data modalities, such as text, images, audio, and structured data, to improve the performance of machine learning models. Given the

heterogeneity and complexity of multi-modal data, pre-processing and feature selection play a critical role in ensuring high-quality data inputs and efficient learning. Effective pre-processing and feature selection techniques can reduce noise, enhance meaningful patterns, and improve computational efficiency [4].

Data pre-processing in multi-modal learning begins with data cleaning and normalization. Since different modalities may have distinct structures and scales, it is essential to standardize data formats and handle missing values appropriately. One modality of data often requires resizing, normalization, and augmentation, while another modality of data needs tokenization, stop-word removal, and vectorization. Similarly, some data require transformations such as noise reduction and spectrogram conversion, whereas structured numerical data may require outlier removal and normalization. The alignment of different modalities, known as data fusion, is another crucial step that ensures consistency across data sources. Techniques such as early fusion (concatenating raw features), late fusion (integrating model predictions), and hybrid fusion (combining both approaches) are widely employed for synchronizing multi-modal data [47].

In the Table 2.2, a comparison of different scaling techniques is given. This scaling technique has a different approach for scaling the data and the formula is given, which shows a method to calculate the scaled data points.

Table 2.2: Comparison of Different Scaling Techniques

Scaling Technique	Methodology	Formula
Min-Max Scaling	Normalization	$Xnew = \frac{X - \text{Min}}{\text{Max} - \text{Min}}$ (2.1)
Standard Scaling	Z-Score Normalization	$Xnew = \frac{X - \text{Mean}}{\text{Standard Deviation}}$ (2.2)
Robust Scaling	Scaling by Median and IQR	$Xnew = \frac{X - \text{Median}}{\text{IQR}}$ (2.3)

Table 2.3 shows the usage of scaling techniques under different conditions. While working on Multi-modal learning, data from different modalities have different ranges, sizes and forms. The same scaling technique is not always applicable; the choice of scaling technique should be based on the type of data. The use of the scaling technique based on its use is given in Table 2.3.

Feature selection is a vital step in multi-modal learning to eliminate redundant or irrelevant fea-

Table 2.3: Usage of different Scaling Technique

Scaling Technique	Usage
Min-Max Scaling	Features with varying ranges bring to a uniform scale
Standard Scaling	Different units or scales, comparison on a standardized scale, less sensitive to outliers than Min-Max Scaling
Robust Scaling	When the dataset contains outliers, robust to outliers and variations in the data

tures while retaining the most informative attributes. Feature selection techniques, such as filter, wrapper, and embedded methods, can be adapted to multi-modal learning. Filter methods, such as correlation-based and mutual information-based approaches, assess feature relevance independently of the learning model. Wrapper methods, such as recursive feature elimination (RFE), iteratively evaluate subsets of features to find the optimal combination. Embedded methods, which integrate feature selection into model training (e.g., Lasso regression and tree-based feature importance), provide a balance between computational efficiency and predictive power [21]. Moreover, deep learning-based methods leverage attention mechanisms and autoencoders to extract high-level feature representations from multi-modal data.

Data pre-processing and feature selection are fundamental components of multi-modal learning that significantly impact model performance. By employing appropriate pre-processing steps and feature selection techniques, it is possible to enhance the quality of input data, reduce computational complexity and improve model generalization [47].

2.9 Short introduction of Transformer Model

In this section, a detailed overview of Transformers architecture, its working, Multi-modal transformer and Encoder-only transformer is given.

2.9.1 Architecture of the Transformer Model

The transformer model consists of an encoder-decoder structure that relies on self-attention and positional encoding instead of recurrence. Figure 2.5 shows different components of Transformer model. This component and their working is explained in the following section.

Encoder-Decoder Structure

The transformer model follows an encoder-decoder structure. The encoder processes the input

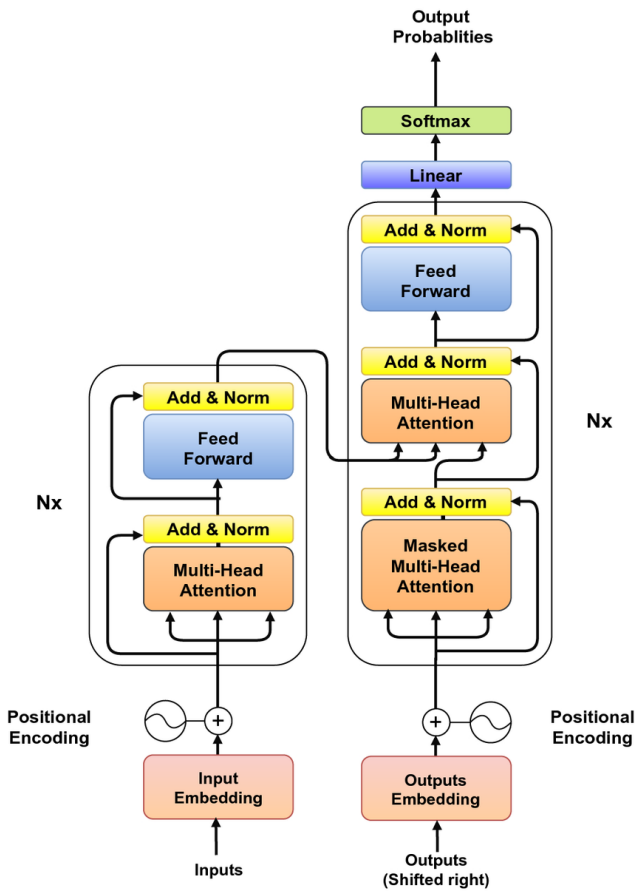


Figure 2.5: Transformer Architecture
[2]

sequence into contextualized embeddings, while the decoder generates output sequences based on the encoder's representations. The encoder consists of multiple layers, each containing self-attention and feed-forward layers. The decoder has similar layers but also includes cross-attention mechanisms to incorporate encoder outputs.

Self-Attention Mechanism

The self-attention mechanism allows the model to assign different weights to different words in an input sequence, enhancing contextual understanding.

The attention scores are computed using the scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.4}$$

where Q, K, and V represent the query, key, and value matrices, respectively, and d_k is the dimension of the key vector.

Multi-Head Attention

Figure 2.6 shows the structure of Multi-head attention. It extends self-attention by allowing the model to jointly attend to information from different representation subspaces. Instead of computing attention once, the model splits queries, keys, and values into multiple heads, performs attention separately, and then concatenates the results.

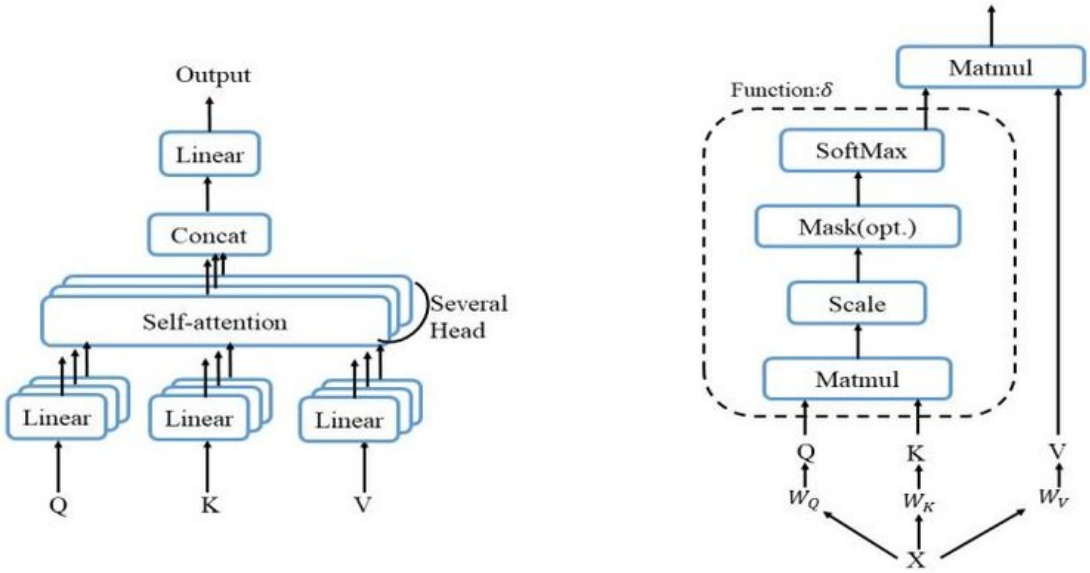


Figure 2.6: Multi-Head Attention Mechanism
[38]

Feed-Forward Networks (FFN)

Each encoder and decoder layer has a position-wise Feed-Forward Networks (FFNs), which consists of two linear transformations with a rectified linear unit (ReLU) activation function:

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2 \tag{2.5}$$

where W_1, W_2 are weight matrices and b_1, b_2 are bias terms. Figure 2.7 shows the structure of a typical feed-forward network.

Positional Encoding

Unlike recurrent neural networks (RNNs), transformers do not process sequences in order, necessitating positional encodings to retain sequence information. Positional encodings use sine and cosine functions to provide unique position information to each token. The working of positional encoding is explained with an example given in Figure 2.8.

$$P(k, 2i) = \sin\left(\frac{k}{n^{2i/d}}\right) \tag{2.6}$$

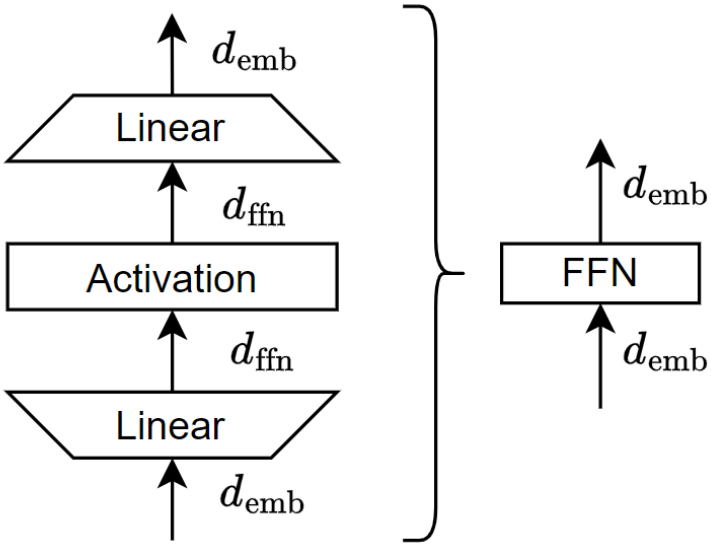


Figure 2.7: A typical feed-forward network
[38]

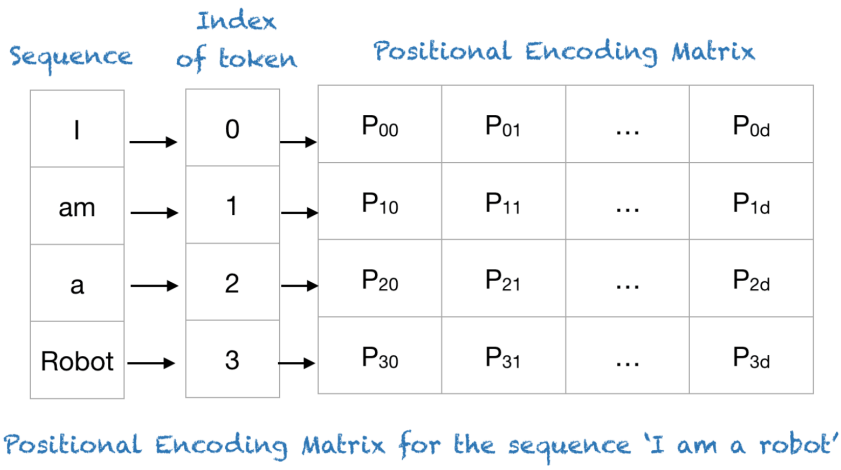


Figure 2.8: An example of Positional Encoding
[38]

$$P(k, 2i+1) = \cos\left(\frac{k}{n^{2i/d}}\right) \tag{2.7}$$

Here:

- k: Position of an object in the input sequence, $0 \leq k < L/2$
- d: Dimension of the output embedding space
- $P(k, j)$: Position function for mapping a position k in the input sequence to index (k, j) of the positional matrix
- n: User-defined scalar, set to 10,000 in [41].

- i: Used for mapping to column indices $0 \leq i < d/2$, with a single value of i mapping to both sine and cosine functions

In the above expression, you can see that even positions correspond to a sine function and odd positions correspond to cosine functions. The working mechanism is given in Figure 2.9

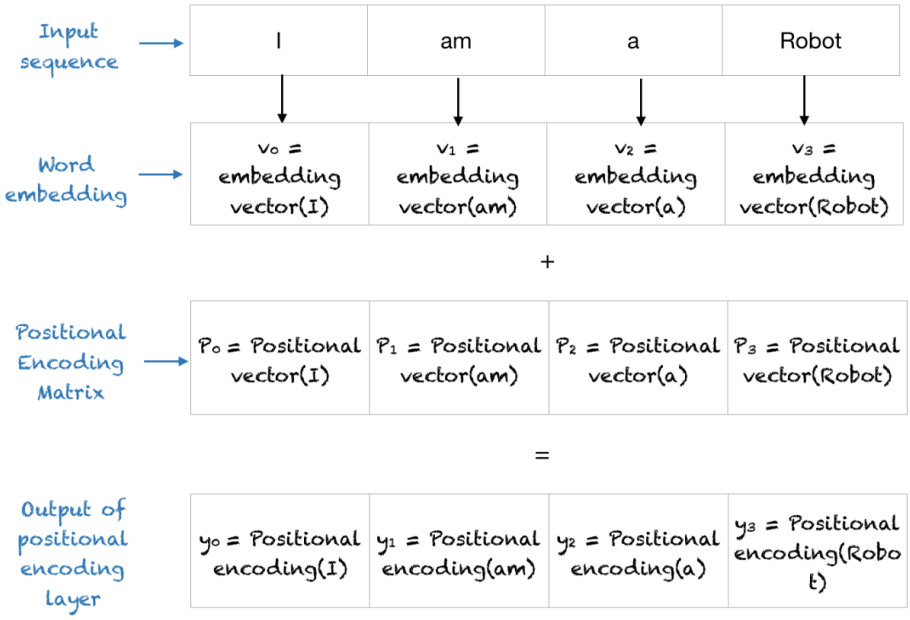


Figure 2.9: Working mechanism of Positional Encoding with word embedding [38]

2.9.2 Multi-Modal Transformer

Multi-modal transformers integrate multiple data modalities, such as text, images, and audio, within a unified architecture.

Structure of Multi-Modal Transformer

The multi-modal transformer consists of multiple encoders and decoders, each tailored to process a specific data modality. Figure 2.10 shows a possible structure of a multi-modal transformer. This structure generally includes:

- **Modality-Specific Encoders**: Each modality, such as text, images, or audio, is processed using a dedicated encoder. These encoders transform raw inputs into high-dimensional embeddings suitable for cross-modal interactions.
- **Cross-Modal Attention Mechanisms**: Multi-modal transformers employ cross-attention mechanisms to allow interactions between different modalities. This mechanism enables the model to align and fuse information from multiple input sources.

- **Fusion Layer**: The embeddings from different encoders are combined in a fusion layer, which integrates the multi-modal features through techniques like concatenation, summation, or attention-based alignment.
- **Unified Decoder**: The decoder processes the fused multi-modal representations to generate outputs, such as captions for images, transcriptions for audio, or multi-modal reasoning tasks. The decoder applies attention to both the fused representations and the original input embeddings to enhance contextual understanding.
- **Modality-Specific Normalization and Positional Encoding**: Since different modalities have varying characteristics, normalization layers ensure that feature representations remain coherent. Additionally, positional encoding is extended to capture spatial and temporal dependencies in vision and speech modalities.

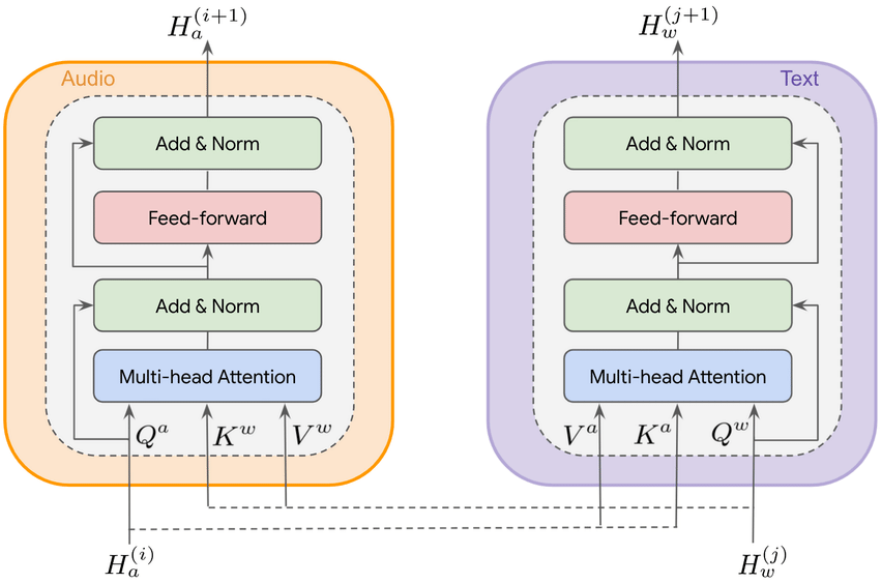


Figure 2.10: Structure of Multi-modal Transformer [26]

This structured approach enables multi-modal transformers to effectively model relationships across different data types while preserving individual modality-specific information.

2.9.3 Encoder-Only Transformer

The encoder-only Transformer is a neural network architecture designed for representation learning in NLP tasks. Unlike the full Transformer model, which includes both an encoder and a decoder, the encoder-only Transformer focuses solely on understanding input sequences without generating new ones. This architecture is widely used in models such as Bidirectional Encoder Representations from Transformers (BERT) for tasks like text classification, named entity recognition, and

embedding generation [9].

Working of Encoder-Only Transformer

The encoder-only Transformer operates by processing input text through multiple layers of self-attention and feed-forward mechanisms to generate contextualized embeddings [41]. The key steps involved in its operation include:

Tokenization and Embedding: The input text is divided into tokens and mapped to dense vector representations.

Adding Positional Encoding: Since self-attention does not inherently capture word order, positional encodings are added to retain sequential information.

Multi-Head Self-Attention Processing: The model captures dependencies between tokens, enabling contextual understanding.

Feed-Forward Processing: Each token representation is refined through a non-linear transformation to enhance expressiveness.

Layer Normalization and Residual Connections: These techniques help stabilize training and improve gradient flow.

Stacking Multiple Encoder Layers: The model stacks several layers to extract deep semantic meaning from input text.

Final Output Extraction: The resulting embeddings are utilized for downstream tasks such as classification, information retrieval, and question answering.

Usage for Classification in Sequential Data

Encoder-only Transformers are particularly effective in sequential data classification, where context is crucial for accurate predictions. Some key applications include: In sentiment analysis, spam detection, and topic classification, the encoder-only Transformer processes text and generates embeddings, which are then used for classification [9]. Tasks such as named named entity recognition (NER) rely on the model's ability to generate context-aware representations. Beyond NLP, encoder-only Transformers are also adapted for structured sequential data, such as financial trend analysis, stock price prediction, and health monitoring, where long-term dependencies are essential [10]. In medical research, these models assist in document classification, entity extraction, and literature review automation. Encoder-only Transformers are utilized in automatic speech recognition pipelines for the task of speech emotion recognition. These models help classify and detect anomalous behavior in sequential logs and transaction data.

The effectiveness of encoder-only Transformers in classification tasks stems from their ability to model long-range dependencies and contextual relationships between input tokens. This makes them highly suitable for processing sequential data across different domains.

The encoder-only Transformer is an essential architecture for tasks requiring deep language understanding. Its ability to model complex dependencies through self-attention makes it highly effective [41]. By focusing exclusively on encoding input sequences, it optimizes performance for tasks that do not require text generation. Furthermore, its success in classification tasks across domains highlights its versatility in processing sequential data [10].

3 State of The Art

In this chapter, we discuss various available models for Multi-modal classification, challenges for selecting a suitable model and a comparison of different models for multi-modal classification tasks.

3.1 Available Models for Multi-modal Classification

Multi-modal classification refers to the task of integrating and analyzing multiple types of data collected from various sources to make accurate predictions. There has been no approach to using S-parameter and TDR data with a multi-modal classification model for failure detection. However, the components of multi-modal classification can be adapted for our thesis. Some of the models used for multi-model classification are given below, although the use case is different from our project. But, for the selection of a suitable model, we will compare these models based on the nature of S-parameter and TDR dataset and some challenges involved with this dataset.

- **Autoencoders:** Autoencoders are unsupervised neural networks designed for representation learning by compressing and reconstructing input data. In multi-modal classification, autoencoders are utilized to extract meaningful features from heterogeneous data sources by learning a shared latent representation that captures cross-modal correlations. Variants like VAEs and multi-modal deep autoencoders (MDAEs) enhance the learning process by enforcing structured latent spaces and improving modality fusion. These models facilitate dimensionality reduction while preserving essential information for classification tasks [19].
- **Convolutional Neural Networks (CNNs):** CNNs are widely used for feature extraction in multi-modal classification due to their ability to capture spatial hierarchies in structured data. CNNs process modalities like images and time-series signals by learning local and global patterns through convolution operations. When combined with other architectures, such as RNNs or attention mechanisms, CNNs can efficiently integrate multiple modalities, ensuring robust feature extraction. Additionally, multi-modal CNNs incorporate modality-specific branches to learn independent feature maps before fusing them into a unified representation for classification [42].
- **Transformers:** Transformer models, originally designed for sequential data processing, have

demonstrated superior performance in multi-modal classification by leveraging self-attention mechanisms. They enable the fusion of diverse modalities by learning long-range dependencies and contextual relationships across modalities. Recent transformer-based architectures, such as Vision Transformers and Multi-modal Transformers, employ tokenization strategies to represent different data types uniformly, allowing for seamless modality integration. These models improve classification by dynamically weighting relevant features and mitigating modality imbalance [4].

- **Hybrid Models (CNN + LSTM, Transformer + LSTM):** Hybrid models, such as CNNs - Long short-term memory (LSTM) and Transformer-LSTM architectures, play a crucial role in multi-modal learning by effectively capturing both spatial and temporal dependencies across different data modalities. CNNs excel at extracting spatial features from visual inputs, while LSTM networks capture sequential dependencies, making CNNs - LSTM a powerful combination for tasks like action recognition from videos. Similarly, Transformers are effective in learning long-range dependencies and contextual relationships, but integrating them with LSTM can enhance their ability to model temporal sequences in a structured manner. This fusion allows the model to dynamically adapt to different types of sequential and non-sequential inputs, improving classification performance in complex multi-modal applications [4].
- **LSTM and Gated Recurrent Unit (GRU):** LSTM and GRU are widely used in multi-modal learning to handle sequential dependencies across different data modalities. These recurrent architectures are designed to mitigate the vanishing gradient problem, enabling efficient learning of long-range dependencies. While LSTM use separate forget, input, and output gates to regulate information flow, GRU simplifies this mechanism with a combined update and reset gate, making them computationally more efficient. In multi-modal tasks, LSTM and GRU help align and fuse information from different data sources by learning temporal correlations within each modality. For example, in audiovisual emotion recognition, LSTM can capture temporal patterns in speech, while GRU processes video features, enabling the model to integrate insights from both modalities. The choice between LSTM and GRU depends on task complexity, with GRU being faster and requiring fewer parameters, while LSTM offer better control over long-term dependencies [8].

Each of these models offers unique advantages depending on the nature of the data and the classification task. The choice of model should be based on the level of complexity, computational resources, and the specific requirements of the application.

3.2 Selection of Suitable model for Multi-modal classification

In section 3.1, a short description of available models for the multi-modal classification task is given. These models are well suited for the multi-modal data and the adaptability of these models with changing data. In this section, we will discuss the suitability of these models according to our dataset and our failure classification task.

Before starting with the suitability and selection of the model, let's discuss a few observations about the nature of the data and challenges faced while working on this data. These factors are crucial in deciding the selection of the model for the failure classification task. Further information about the S-parameter and TDR data and the creation of the final dataset is discussed in section 4.1.

3.2.1 Sequential data

S-parameter and TDR data for a GaN HEMT transistor are essential for characterizing its high-frequency performance and transient response. S-parameter data provide insights into the transistor's impedance matching, gain, and stability across different frequencies. TDR data obtained through a time-domain reflectometer or equivalent setup helps analyze signal integrity, detect impedance mismatches, and evaluate the transistor's parasitic effects. Sequentially writing these datasets ensures a comprehensive understanding of the transistor's behavior.

Sequential data refers to data where the order of elements matters. Unlike regular tabular data, where individual rows are independent, sequential data has a structure where past elements influence future elements. This type of data appears in various real-world applications, including time series forecasting, NLP, and speech recognition [39].

Examples of Sequential Data in the field of Power electronics and Sensor data:

Switching Waveforms: Inverters, rectifiers, and DC-DC converters generate sequential voltage and current waveforms over time, which can be analyzed for fault detection or efficiency optimization [28].

Harmonic Distortion Signals: Power quality monitoring systems record harmonic distortions in voltage and current waveforms, which are sequential data used for diagnosing grid issues [5].

Temperature Time-Series: Thermal sensors in power converters or transformers provide sequential temperature data for thermal management and failure prediction [50].

Current and Voltage Sensor: Smart meters and IoT-based power monitoring systems record real-time current and voltage variations, forming sequential datasets used for load forecasting and

anomaly detection [11].

3.2.2 Challenges of Sequential Data in Classification

- **Temporal Dependencies:** One of the most significant challenges in classifying sequential data is the presence of temporal dependencies, where past elements influence future ones. This characteristic makes it difficult to model and analyze sequences since each element's meaning is often context-dependent. Unlike independent and identically distributed (i.i.d.) data, sequential data exhibits correlations over time, making traditional classification methods less effective. For instance, in financial time series analysis, the value of a stock today is influenced by its historical values, and ignoring these dependencies can lead to poor predictions. Similarly, in speech recognition, phonemes depend on preceding sounds, and misinterpreting earlier parts of the sequence can distort the entire prediction. The challenge lies in determining the extent to which past elements should influence current decisions, as sequences often contain both short-term and long-term dependencies, which vary across different applications [13].
- **Variable-Length Sequences:** Another challenge in sequential data classification is handling sequences of varying lengths. Unlike standard classification tasks where inputs are typically of fixed dimensions, sequential data can be highly variable in length, making it difficult to process and compare sequences directly. For example, sentences can range from a few words to several paragraphs, requiring flexible handling of inputs. Sequences can differ significantly in length, posing difficulties for feature extraction and alignment. The variability in sequence length also complicates training models, as they must be able to generalize across sequences of different sizes without being biased toward shorter or longer ones. Furthermore, sequences often contain padding or truncation when processed in batches, which can introduce noise and affect classification performance [3].
- **Feature Engineering:** Extracting meaningful features from sequential data is a crucial but challenging aspect of classification. Unlike structured tabular data, where features are predefined and static, sequential data often contains redundant, noisy, or irrelevant information. Identifying the most informative patterns requires domain expertise and sophisticated pre-processing techniques. For instance, in speech and audio classification, raw waveforms contain vast amounts of information, but only specific frequency components may be relevant for distinguishing between classes. Sequences must be transformed into numerical representations while preserving their meaning. The challenge intensifies in multimodal sequences, information where it is spread across multiple time frames and modalities. Poorly engineered features can lead to suboptimal classification performance, highlighting

the complexity of working with sequential data [34].

3.2.3 Challenges for Multi-modal Classification

Multi-modal classification, which involves integrating information from diverse data sources, presents unique challenges that can hinder model performance. Key issues include:

- **Overfitting Due to Increased Model Capacity:** Combining multiple modalities often leads to models with higher complexity, increasing the risk of overfitting, especially when training data is limited. This complexity can cause the model to perform well on training data but poorly on unseen data [44].
- **Limited Availability of Large, Labeled Datasets:** Effective training of deep learning models typically requires large datasets. However, in multi-modal contexts, such datasets are often scarce, particularly when involving specialized data like medical images. The lack of extensive, labeled datasets hampers the development of robust multi-modal models [37].
- **Class Imbalance:** Multi-modal datasets frequently exhibit class imbalance, where certain classes are underrepresented. This imbalance can bias the model toward majority classes, reducing its ability to accurately predict minority classes. Addressing this requires specialized techniques beyond standard oversampling or undersampling methods [44].
- **Instance-Level Difficulty:** Some data instances are inherently more challenging to classify due to their proximity to decision boundaries or the presence of noise. In multi-modal settings, this issue is compounded as each modality may contribute its own set of complexities, making certain instances particularly difficult to classify accurately [37].
- **Integration of Heterogeneous Data:** Effectively fusing information from different modalities, each with its own data structure and statistical properties, remains a significant challenge. Inadequate fusion strategies can lead to suboptimal exploitation of the available information, thereby limiting model performance [48].

3.2.4 Comparison of different Models for the Multi-modal classification

Multi-modal classification involves integrating and analyzing sequential data, making the choice of model crucial for accurate predictions. As discussed in Sections 3.1 and 3.2, different models have varying capabilities in handling sequential data challenges such as temporal dependencies, variable sequence lengths, and feature extraction. This section compares Autoencoders, CNNs, and Transformers to determine the most suitable model for our task.

From the comparison, Autoencoders are useful for feature extraction but are not designed for classification, making them less suitable for handling sequential dependencies. CNNs performs

Table 3.1: Comparison of models for Multi-modal classification

Challenges	Autoencoders	CNNs	Transformers
Temporal Dependencies	Weak, as autoencoders primarily focus on feature extraction rather than sequential relationships.	Weak, since CNNs primarily extract spatial features and struggle with sequential dependencies [20].	Excellent, as self-attention mechanisms capture both short- and long-range dependencies efficiently [41].
Variable Sequence Lengths	Limited since autoencoders work best with fixed-length data representations [36].	Struggles with variable sequence lengths, requiring padding or resizing [7].	Naturally handles variable-length sequences due to parallel processing and self-attention [18].
Feature Extraction	Strong in feature learning but lacks direct interpretability for sequential dependencies [12].	Strong in spatial feature extraction but inefficient for sequential data [9].	Excels in automated feature extraction across multiple modalities using attention mechanisms [10].
Computational Efficiency	Efficient for dimensionality reduction but not optimized for classification [14].	Highly efficient due to parallelized convolution operations [14].	Efficient in handling complex data, though memory-intensive [6].

well in spatial feature extraction but cannot model sequential relationships effectively. LSTM and GRU can handle sequential data but struggle with long-range dependencies and computational inefficiency.

Transformers, in contrast, outperform these models in handling sequential data challenges. Their self-attention mechanisms allow them to efficiently model both short- and long-term dependencies, making them particularly well-suited for multi-modal classification tasks involving S-parameter and TDR data. Additionally, their ability to process sequences in parallel enhances computational efficiency compared to recurrent architectures.

Given the complexity of multi-modal classification, Transformers emerge as the most suitable choice for our task due to their ability to capture sequential relationships, adapt to varying sequence lengths, and efficiently extract meaningful features.

4 Implementation

After considering the related work and concepts in Chapter 2 and the comparison of various state-of-the-art models in Chapter 3. In this chapter, the application of these concepts in the design process of the multi-modal classification model is presented in detail. The overall approach of the given task is explained. The first step of the multi-modal classification model is to prepare the dataset in a form that is suitable for the model. As already discussed the purpose of creating a classification model is for failure classification. The available data and its nature have to be studied before starting with the classification task. Also, the steps taken to prepare the dataset and the implemented steps are explained in this section.

Although the scope of the thesis is to develop a multi-modal model but uni-modal model is also developed. This model acts as a base model in the development of the multi-modal model and this model's performance acts as a reference for improvement in the main multi-modal model. Also, certain transformer model parameters are set to for the optimum model performance in the uni-modal model itself. Considering this, there is a section of the uni-modal transformer model and its working is explained in this chapter.

4.1 Dataset Description

For GaN HEMT transistor, we are going to use S-parameter and TDR data for the failure classification. In this section, we discuss the creation of a uniform dataset from the raw data files and important parameters for which the dataset is defined. Also, plots of S-parameter and TDR plots are shown for DUT 1 and 5.

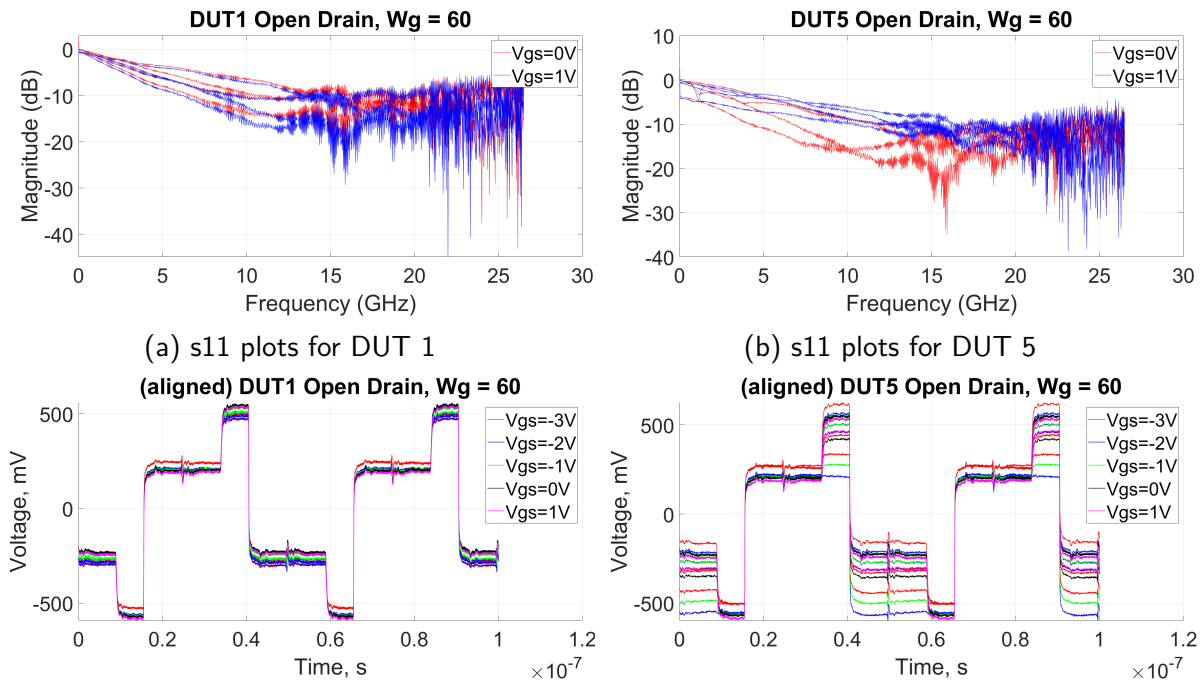
4.1.1 s11 and TDR plots for DUT 1 and 5

The figure 4.1 presents s11 and TDR plots for DUT 1 and DUT 5. The s11 plots (a and b) indicate the reflection coefficient across a frequency range, showing how much signal is reflected due to impedance mismatches, which is crucial in assessing signal integrity and impedance matching. The TDR plots (c and d) display the voltage response over time, helping to identify impedance discontinuities along the transmission path.

Table 4.1: DUT value definition

DUT value	Explanation	DUT value (in Dataset)
1	Failed transistor	0
5	Working transistor	1

In this context, the DUT values and their explanations are given in table 4.1. The DUT value signifies the electrical performance characteristics, including impedance, signal integrity, and consistency across different devices. Variations in these plots can indicate design inconsistencies, material defects, or process variations affecting device reliability, with DUT 1 highlighting failure-related anomalies and DUT 5 serving as a reference for the working condition of the transistor.



(a) s11 plots for DUT 1

(b) s11 plots for DUT 5

(c) TDR plots for DUT 1

(d) TDR plots for DUT 5

Figure 4.1: s11 and TDR plots for DUT 1 and 5

4.1.2 S-parameter and TDR Dataset

The S-parameter dataset is generated from .s1p files, which contain the s11 parameters of the DUT. These files store three columns: Frequency, Real(s11), and Imag(s11). The dataset consists of measurements at 1000 distinct frequency points, ranging from 26.53 MHz to 2650 MHz. However, for this study, only the real component of the s11 parameter is considered.

Each .s1p file is defined based on four key parameters: V_{ds}, V_{gs}, L_{g}, and the DUT. The values assigned to these parameters are listed in Table 4.1. By combining these parameter values, a total

of 32 unique s11 samples are obtained, each comprising 1000 real(s11) data points. Notably, only positive values are considered in this study. Initially, the model performance is evaluated with positive values and the absolute value of s11. The model showed no changes or improvement in the result with both types of values. Therefore, for simplicity and less computation, only real(s11) data points are considered.

Table 4.2: Values of key parameters

Parameter	Values
V_{ds}/V	4 V, Open
V_{gs}/V	-3, -2, -1, 0, 1
L_{g}	0.25, 0.50, 0.75, 1.00
DUT	1 and 5

Similar to the S-parameter dataset, a TDR dataset is also constructed. The TDR samples are defined based on the same key parameters listed in Table 4.2. In total, the dataset consists of 75 unique TDR samples. Each sample contains 63,000 data points, which are extracted from corresponding .mat files.

The TDR dataset provides time-domain signal reflections, which provide information about the working condition of the transistors. This information is summarized by DUT values. By analyzing both datasets, a comprehensive understanding of the complete data is achieved.

4.2 Data pre-processing and Synchronization

The `creating_sync_dataset` function synchronizes datasets derived from s11 measurements and TDR measurements. The function processes .s1p files containing s11 parameter data and .mat files containing TDR data, extracts relevant parameters, and aligns these datasets based on key parameters given in table 4.1. This synchronization ensures a reliable and structured dataset, which is crucial for failure classification of DUT.

The function takes two inputs:

- `S1P_file_path`: Directory containing .s1p files.
- `TDR_file_path`: Directory containing .mat files.

The function first processes the s11 data by extracting s11 values from .s1p files and using the first file's frequency values as a reference. It then creates a dataset where each row corresponds to a DUT file, and columns represent frequency-dependent s11 values. A DUT column is added based on the filename, where DUT=1 is labeled as 1 and DUT=5 as 0, and key parameters such as L_{g}, V_{ds} and V_{gs} are extracted from filenames.

For TDR data, the function loads .mat files, extracts voltage waveform data, renames columns to

represent time-based measurements, and merges them with metadata from an Excel file containing key parameters. It combines them into a single dataset, `TDR_dataset`, ensuring that the DUT column is moved to the last position for better readability. Once both datasets are created, the function synchronizes them by parameters common samples based on shared columns (L_{g}, V_{ds}, V_{gs} and DUT). It generates the following datasets given in Table 4.3.

Table 4.3: Values of key parameters

Dataset	Description	Number of Samples
`sync_s11_dataset`	Matching s11 data	30
`sync_TDR_dataset`	Matching TDR data	30
`s11_dataset`	Entire s11 data	32
`TDR_dataset`	Entire TDR data	75

This function is essential for data pre-processing and synchronization in failure classification. By aligning frequency-domain s11 measurements with time-domain TDR waveforms, it provides a structured dataset that aids in evaluating device performance, modeling behavior, and analyzing failures.

4.3 Data Scaling and splitting

The function `data_Scaling_Sync` is designed to process and scale synchronized datasets, specifically `sync_s11_dataset` and `sync_TDR_dataset`, while also preparing them for use in machine learning models. The function begins by defining fixed dataset sizes for training, testing, and validation. It then randomly shuffles the `sync_s11_dataset` using a predefined random state to ensure reproducibility. The dataset is split into training and temporary data subsets, where the temporary subset is further divided into test and validation sets. To maintain class balance in the test dataset, an equal number of samples from each class (DUT 0 and 1) are selected before shuffling again.

4.3.1 Dataset Synchronization

To ensure that the TDR dataset matches the s11 dataset, a filtering function, `filter_dataset_by_columns`, is applied based on specific matching columns: `Lg`, `Vds`, `Vgs`, and DUT. This filtering ensures that only relevant data points are considered in further processing. Subsequently, inner merges are performed between the respective s11 and TDR subsets to retain only the rows where both datasets have matching values for the defined columns. After merging, the datasets are separated back into their original structures, ensuring that s11 and TDR subsets remain distinct but synchronized.

4.3.2 Data Scaling and Tensor Conversion

Before scaling the dataset, the length of each TDR sample is reduced. This step helps in reducing noise and improving the stability of the TDR dataset. Following this, the feature columns are extracted separately from the target variable (`DUT`) for both s11 and TDR datasets, resulting in and variables for each subset (training, validation, and test). The ratio for splitting is 60% for training, 20% for testing and 20% for the validation set. For data scaling, the function offers multiple scaling techniques, including Standard Scaling, Min-Max Scaling, Robust Scaling, and Max-Abs Scaling. One of the scaling method is selected through the `scaling_type` parameter, and the corresponding scaler is applied to transform the feature data. Standardization is performed separately for s11 and TDR datasets to avoid data leakage. The transformed datasets are then converted into PyTorch tensors, ensuring compatibility with deep learning frameworks.

4.3.3 Dataset Preparation for Model Training

Once converted into tensors, the datasets are wrapped into PyTorch's `TensorDataset` objects, allowing for efficient handling during model training. The function then creates `DataLoader` instances for each subset, including training, validation, and test sets, with a batch size of 4. The `shuffle` option is enabled for training data to ensure random sampling in each epoch, promoting better generalization in the model. Finally, the function returns six data loaders corresponding to the processed s11 and TDR datasets, ready for training and evaluation within a machine learning pipeline.

4.4 Setup for Model Training

The training function is designed to train a deep learning model while preventing overfitting using early stopping. The function starts by setting up variables to track the best validation loss and the number of epochs without improvement. It then runs a loop for training, where the model is set to training mode, and batches of data are taken from the data loaders. Each batch contains data from two sources, s11 and TDR datasets. The data and labels are moved to the right device, and the model is trained using backpropagation. The optimizer updates the model's parameters based on the loss, which is calculated using a predefined method. The training accuracy is checked by comparing predicted labels with actual labels.

During the training process, the model performance is simultaneously evaluated on validation data to see how well the model training is done. In this step, calculations are done without updating gradients to save resources. The validation loss is recorded for each batch, and predictions are

compared to actual labels to calculate accuracy. Early stopping is used by checking if the validation loss improves. If the loss does not improve for a set number of epochs, training stops to avoid overfitting, and the best validation loss is saved.

The testing function works similarly but is only used to check the final model on the test dataset. The model is put in evaluation mode, and test data is passed through it to get predictions. The predicted labels are compared with actual test labels to find accuracy, a confusion matrix, and a classification report. The confusion matrix shows how well the model classified each category, while the classification report gives details about precision, recall, and F1-score. These results help to understand how well the model performs. The test accuracy, confusion matrix, and classification report are displayed to summarize the model's performance. This method ensures that the model is trained well and tested properly.

The chosen training parameters in Table 4.4 are carefully selected to balance model performance, training efficiency, and the risk of overfitting. Setting the number of epochs to 50 provides the model with enough time to learn meaningful patterns without overtraining, especially since early stopping is in place to halt training if no improvement is observed. A small batch size of 4 is used, which is particularly effective when working with limited or high-dimensional data such as s11 and TDR datasets, as it allows for more frequent updates and better generalization while also reducing memory usage. The learning rate of 0.001 is a commonly accepted default that offers a good balance between convergence speed and stability, particularly when used with the Adam optimizer, which adaptively adjusts learning rates for each parameter and performs well in many scenarios, especially with noisy or sparse data. CrossEntropyLoss is the standard loss function for multi-class classification tasks, making it appropriate here as it quantifies the difference between predicted probabilities and true class labels. Early stopping with a patience value of 5 ensures that the model stops training if it hasn't improved on the validation set for five consecutive epochs, helping to avoid overfitting while still allowing for small fluctuations in validation loss. Together, these parameter choices are well-justified for building a stable and accurate deep learning model that can generalize well to unseen test data.

Table 4.4: Training Parameters

Training Parameter	Value
Number of Epochs	50
Batch Size	4
Learning Rate	0.001
Optimizer	Adam
Loss Function	CrossEntropyLoss
Early Stopping Patience	5

4.5 Transformer Model for classification

Now, both the s11 and TDR dataset is prepared and ready to be used. In section 3.2.4, it's established that the transformer model is well suited for this application considering the nature of the dataset and given the task of Multi-modal classification. In this section, we will implement the Transformer model for Single modality(TDR data) and then adapt it for multi-modal application. Also, the implementation of zero-value imputation and transfer learning for the transformer model is explained. This gives an overview of the entire scope of the work in this thesis.

4.5.1 Uni-Modal Transformer Model

In this section, a Uni-modal Transformer model is implemented for the single modality and to evaluate the model performance the a Uni-modal transformer model is created. It can be used to process and classify only 1 modality. Now, as we have already studied the dataset. The TDR dataset is selected for this, considering the large number of samples present in the dataset. Also, even after synchronous samples, there are 45 TDR samples left.

The Single-Modality Classifier is a neural network model implemented using PyTorch, designed for classification tasks based on a single input sequence. Unlike multi-modality models that process multiple sequences, this model focuses on a single sequence of fixed length. The architecture follows a Transformer-based approach, where the input features are first mapped to a higher-dimensional representation using an embedding layer. A Transformer encoder then processes this representation to capture temporal dependencies within the sequence. The transformed output is subsequently flattened and passed through a fully connected classification layer to generate the final class predictions.

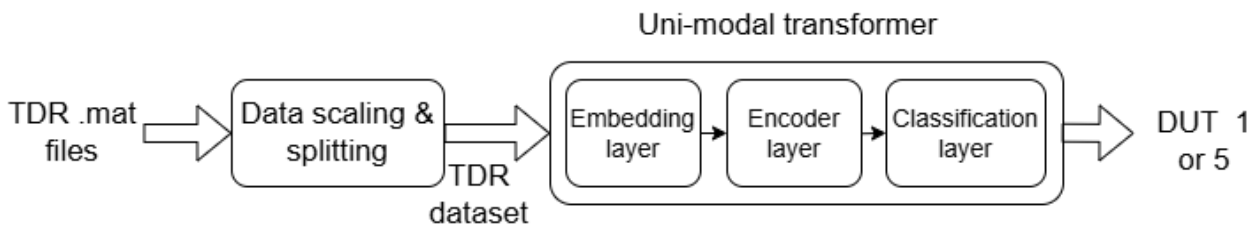


Figure 4.2: Single Modality Transformer Architecture

The model's architecture is shown in Figure 4.2. The `seq_len` parameter specifies the length of the input sequence, while `input_dim` represents the number of features in each sequence element. The `num_classes` parameter determines the number of output classes for classification. The `d_model` parameter, which defaults to 128, defines the size of the feature space used by the Transformer encoder. The nhead parameter, set to 8 by default, represents the number of

attention heads in the Transformer, allowing the model to capture diverse aspects of the input sequence. Additionally, the `num_layers` parameter specifies the number of stacked Transformer encoder layers, enabling deeper representation learning.

The input sequence is first passed through a linear embedding layer (self.embedding), which projects the features from their original `input_dim` space to a `d_model`-dimensional space. The output is then permuted to match the required format for the Transformer encoder, rearranging its shape to [`seq_len`, `batch_size`, `d_model`]. The Transformer encoder layer applies self-attention mechanisms and feed-forward transformations to extract meaningful dependencies within the sequence. After processing, the output is permuted back to [`batch_size`, `seq_len`, `d_model`] to restore the batch-wise structure.

To prepare the data for classification, the transformed sequence is flattened into a tensor of shape [`batch_size`, `seq_len` * `d_model`]. This operation converts the sequential representation into a fixed-size feature vector suitable for classification. The final classification layer, implemented as a fully connected (nn.Linear) layer, maps this representation to `num_classes`, producing logits that correspond to class probabilities. By leveraging the power of Transformer-based encoding, the Single-Modality Classifier effectively captures complex patterns within a single input sequence, making it well-suited for sequence classification tasks.

4.5.2 Multi-modal Transformer Model

The Multi-modal Classifier is a neural network model built using PyTorch, designed for classification tasks involving two separate sequences of input features. It leverages Transformer encoders to process each sequence independently before concatenating them for final classification. The model takes two input sequences, each of different lengths, and maps them to a common feature space using separate embedding layers. These embeddings are then passed through individual Transformer encoder layers to extract meaningful representations. Finally, the processed sequences are concatenated and fed into a fully connected classification layer to produce the final output.

The architecture of the implemented model is shown in Figure 4.3. The `seq_len1` and `seq_len2` parameters specify the sequence lengths of the two input datasets. The `input_dim` parameter represents the number of features in each element of the sequences. The `num_classes` parameter determines the number of output classes for classification. The `d_model` parameter, set to a default of 128, defines the dimensionality of the feature space used by the Transformer. The nhead parameter, with a default value of 8, specifies the number of attention heads in the Transformer encoders, allowing the model to capture different aspects of the input data. Additionally, the `num_layers` parameter determines the number of Transformer encoder layers used for each sequence.

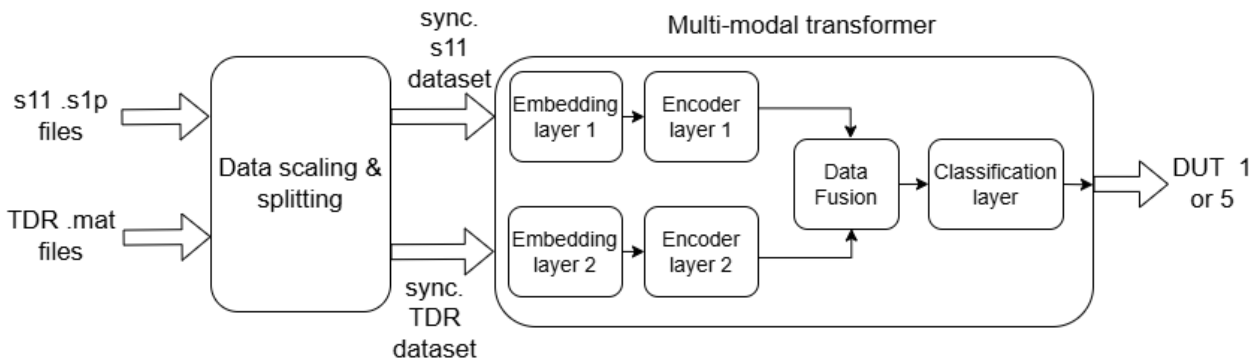


Figure 4.3: Multi-Modal Transformer Architecture

To process the input data, the model first applies linear embedding layers (self.embedding1 and self.embedding2) to project the input features into a `d_model`-dimensional space. These embedded representations are then permuted to match the expected input format of the Transformer encoder layers ([`seq_len`, `batch_size`, `d_model`]). Each sequence is passed through its respective Transformer encoder, which consists of multi-head self-attention and feed-forward layers, enabling the model to learn complex dependencies within the sequences. After encoding, the outputs are permuted back to [`batch_size`, `seq_len`, `d_model`], maintaining the original batch-wise structure.

The outputs of the two encoders are concatenated along the sequence length dimension, resulting in a combined tensor of shape [`batch_size`, `seq_len1` + `seq_len2`, `d_model`]. This tensor is then flattened into a 2D representation of shape [`batch_size`, (`seq_len1` + `seq_len2`) * `d_model`] to prepare it for classification. The final classification layer, implemented as a fully connected (nn.Linear) layer, maps this representation to `num_classes`, producing logits that represent class probabilities. This design allows the model to effectively integrate information from two distinct input sequences and make accurate classification predictions.

4.5.3 Multi-modal Transformer Model with Data imputation

The Multi-modal Classifier model was trained using 75 samples, consisting of s11 and TDR data. However, there was a discrepancy in the number of available paired samples, requiring an imputation strategy to ensure uniformity in the dataset. The specific distribution of data was as follows:

- 30 s11 samples have corresponding TDR samples.
- An additional 45 TDR samples are available without matching s11 data.

Since the model architecture expects equal-sized inputs for both s11 and TDR data, the missing s11 samples were imputed with zero values. This imputation was performed by taking the 45

available TDR samples as a reference, ensuring that each TDR sample had a corresponding s11 sample, even if it was artificially created.

The rationale behind using zero imputation was to maintain data consistency while preventing any artificial bias that might arise from other imputation techniques. Zero values serve as neutral placeholders, ensuring that the model does not infer false relationships from synthetic data while still being able to process the inputs correctly.

This approach helped in standardizing the dataset, allowing the model to learn meaningful representations from the available data without being affected by the imbalance in sample availability. Moreover, by balancing the input sequences, the model could leverage the transformer-based encoding effectively, ensuring that each modality contributed equally to the classification task.

Additionally, this imputation method preserved the overall structure of the dataset without introducing distortions that could arise from statistical estimation or data augmentation techniques. As a result, the model training process remained robust and aligned with the real-world data distribution, maximizing its generalization potential for unseen samples.

4.5.4 Trained Multi-modal Transformer Model

After training the Multi-modal Classifier model on the 75 samples, including the imputed s11 data, the trained model was saved for further use. To adapt the model to a new dataset while leveraging its learned feature representations, a transfer learning approach was implemented. In this process, the previously trained model was reloaded, and all layers except the classifier layer were frozen. Freezing these layers ensured that the model retained the feature extraction knowledge acquired during the initial training phase while allowing the classifier to adjust to new data.

Once the model was prepared with frozen transformer encoders, it was evaluated on a synchronized dataset (sync dataset). The purpose of this evaluation was to assess the model's ability to generalize to a different but related dataset using the learned representations from the initial training. Since the feature extraction layers were already trained on the s11 and TDR data, only the classifier layer required fine-tuning. This approach significantly reduced training time while maintaining high performance by adapting the model's decision boundaries to the new dataset.

5 Results

This chapter presents the results obtained from the Uni-Modal and Multi-Modal transformer models. The primary objective of implementing the 2 transformer models is to show the advantage of using multi-modal data over uni-modal data. The goal is to evaluate the effectiveness of these models based on their accuracy on test dataset and analyze how different dataset configurations affect performance. Also, to evaluate the performance of multi-modal transformer on synchronized s11 and glstdr dataset. The results are structured to compare different dataset sizes and to present the impact of transfer learning.

5.1 Uni-modal transformer performance evaluation

The Uni-modal transformer model performance is evaluated on 2 datasets.

- Synchronized TDR dataset of 30 samples
- Entire TDR dataset of 75 samples

The purpose of using these two datasets is to evaluate the model's performance and based on this analysis, determine the strategy for the Multi-modal transformer. The dataset is split into three groups for training and testing, as shown in Table 5.1.

Table 5.1: Dataset configuration for model training and testing

	Training dataset	Validation dataset	Test dataset
TDR 30 samples dataset	18	6	6
TDR 75 samples dataset	45	15	15

Figure 5.1 illustrates the test accuracy of both models over 10 training cycles. The test results indicate varying levels of accuracy depending on the number of training epochs used. The highest recorded test accuracy for the Uni-modal transformer with the synchronized TDR 30 samples is 83.33%, observed at one instance. For the TDR dataset containing 75 samples, the model exhibited improved average accuracy. The highest recorded test accuracy for this dataset is 86.67%, also achieved in multiple test runs. This improvement in accuracy with an increased sample size suggests that a larger dataset contributes to better model generalization.

Confusion Matrix Analysis for Synchronized TDR 30 Samples

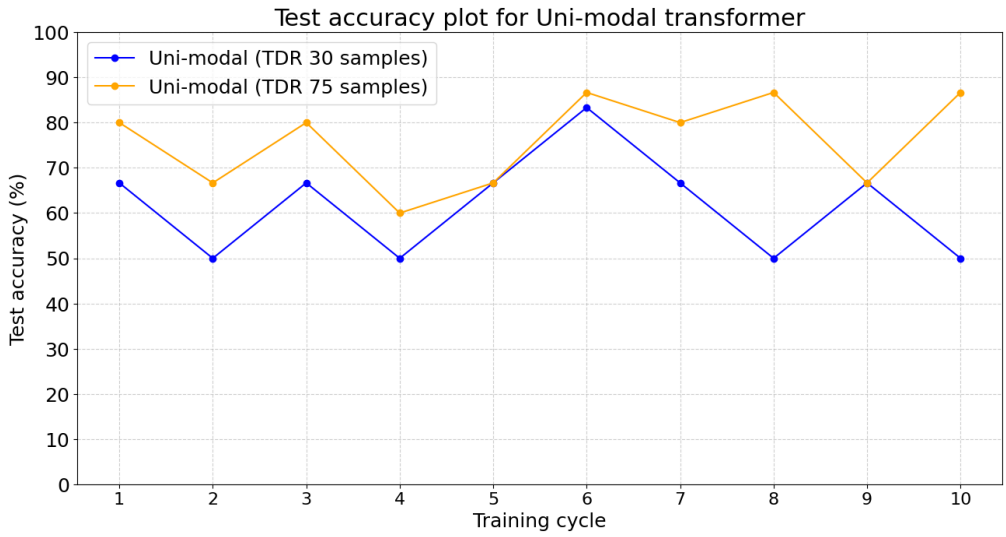


Figure 5.1: Test accuracy results for Uni-modal transformer

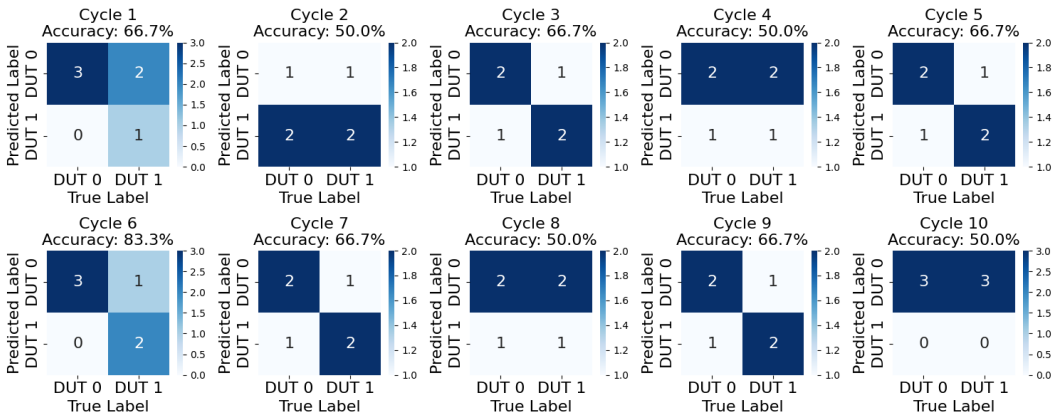


Figure 5.2: Confusion matrix for Uni-modal transformer(TDR 30 samples)

To further analyze the model's predictions, confusion matrices for synchronized TDR data are presented for each training cycle in Figure 5.2. These matrices provide insight into the model's classification performance, illustrating the number of correct and incorrect predictions over 10 training cycles.

In most cases, the model performs well but most of the training cycles the test accuracy fluctuates between 50% and 66.67%. On average, the model achieves 61.67% accuracy on synchronized TDR 30 samples, which suggests that the dataset size may be a limiting factor in achieving higher performance.

The test accuracy improved with increased sample size, suggesting that a larger dataset contributes to better model generalization. Focusing on results of 30 synchronized TDR samples, the model performance is very average, with average test accuracy of 61.67%.

Although model performs very good with TDR 75 samples, the usability of the entire TDR dataset is not possible. The reason is synchronized s11 samples which are limited to 30. This model performance shows that the transformer is suitable for the given dataset. Also transformer models embedding and encoder layer are adequate at feature selection. Now, this model is further modified to adapt s11 data in Multi-modal model. This result of synchronized TDR samples serves as the base for comparison for multi-modal transformer results and the multi-modal models must exhibit improvement in accuracy to prove usability of multi-modal data over uni-modal data.

5.2 Multi-modal transformer performance evaluation

The multi-modal transformer model performance is evaluated on 3 different configurations.

- Synchronized s11 and TDR dataset of 30 samples with Multi-modal transformer model (chapter 4.5.2)
- 30 synchronized + 45 null s11 and 75 TDR dataset with Multi-modal transformer model with modality imputation (chapter 4.5.3)
- Synchronized s11 and TDR dataset of 30 samples with pre-trained Multi-modal transformer model with transfer learning (chapter 4.5.4)

The Figure 5.3 compares the test accuracy of a multi-modal transformer model across above mentioned configurations. The synchronized approach (30 s11 and TDR samples) serves as the baseline, combining both modalities without modifications. The imputation variant (30+45 null s11 and 75 TDR samples) is to handle missing data by artificially introducing null samples while increasing the volume of another modality. The transfer learning configuration (sync. 30 s11 and TDR samples) leverages pre-trained knowledge to improve performance on synchronized s11 and TDR dataset.

Among the three configurations, the multi-modal transformer with transfer learning likely achieves the highest test accuracy, as pre-trained features enhances generalization even with synchronized data. The imputation variant may show competitive performance, particularly if the model effectively handles missing data, but its accuracy could be slightly lower due to introduced noise. The baseline synchronized model, while stable, typically under performs compared to enhanced methods. Thus, transfer learning emerges as the superior approach, balancing robustness and accuracy, while imputation offers a viable alternative for scenarios with incomplete data.

Confusion Matrix Analysis for Synchronized TDR 30 Samples

The analysis of the two confusion matrices highlights the impact of using a pretrained Multi-Modal Transformer model on classification accuracy. The first confusion matrix (Figure 5.4a) represents the results of training the model on synchronized 30 s11 and TDR data, while the

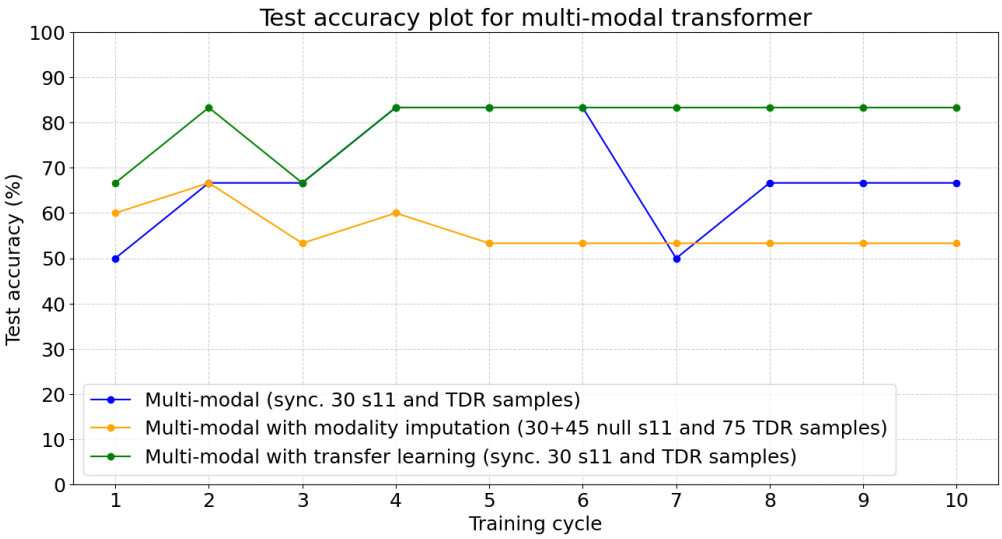


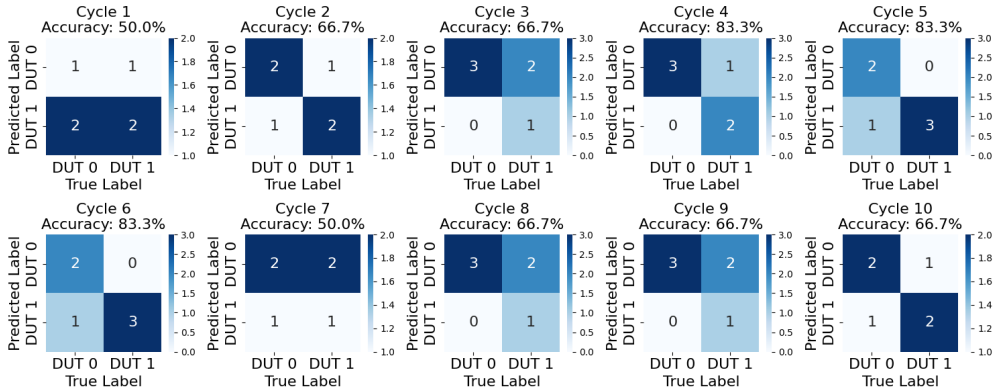
Figure 5.3: Test accuracy results for Multi-modal transformer

second confusion matrix (Figure 5.4b) shows the performance of the same model when pretrained weights were used. A clear improvement in accuracy is observed with the pretrained model, indicating the benefits of transfer learning in this scenario.

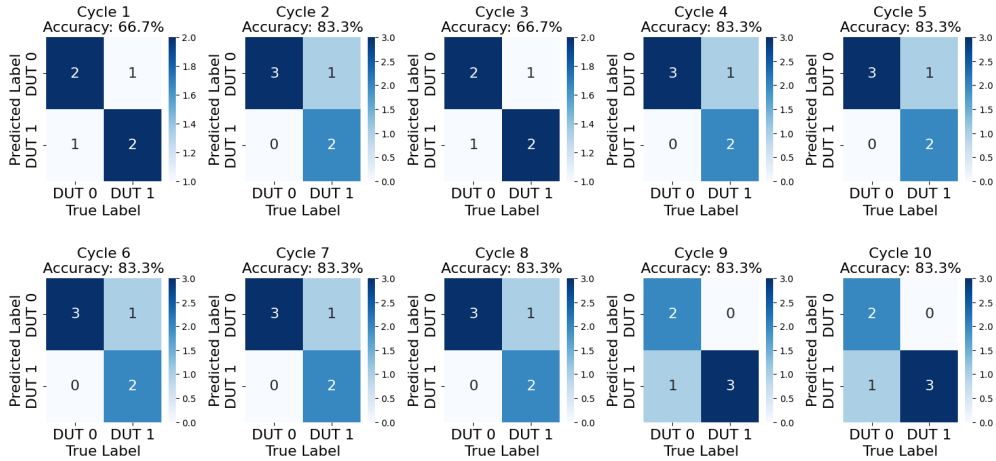
When comparing accuracy across different cycles, the model trained from scratch (Figure 5.4a) exhibits fluctuating performance, with some cycles reaching only 50% accuracy, whereas the pretrained model (Figure 5.4b) demonstrates more stability, consistently achieving 83.3% accuracy in multiple cycles. The reduced mis-classification in Figure 5.4b suggests that transfer learning helped in refining the feature extraction process, leading to better generalization on the dataset.

Another key observation is the reduction in false positives and false negatives in the Figure 5.4b results. The pretrained model seems to have learned more robust feature representations, allowing it to distinguish between classes more effectively. In contrast, the model trained from scratch shows inconsistencies, likely due to a lack of sufficient training data or the inability to learn strong representations from the given dataset alone.

The multi-modal transformer enhanced with transfer learning achieves the highest test accuracy, demonstrating the value of leveraging pre-trained knowledge in multi-modal systems. Its superior performance underscores the importance of feature reuse and adaptation, especially in scenarios with constrained training data. The imputation-based model, while slightly less accurate, remains a strong contender for applications where missing data is inevitable, as it balances robustness with acceptable performance. The synchronized baseline, though straightforward, serves as a critical benchmark but is often outperformed by more sophisticated approaches.



(a) Confusion matrix for Multi-modal (sync. 30 s11 and TDR samples)



(b) Confusion matrix for Multi-modal with transfer learning (sync. 30 s11 and TDR samples)

Figure 5.4: Confusion matrix for Multi-modal transformer model

5.3 Comparative analysis

The accuracy comparison of different model configurations is shown in Figure 5.5. This highlights several key insights. First, the Uni-Modal approach shows improved performance with a larger dataset. Specifically, the model trained with 75 TDR samples achieves an average accuracy of 76%, compared to just 61.67% when trained with only 30 samples. This suggests that the model benefits from more training data, likely because the increased dataset provides better representation of underlying patterns, which helps the model generalize more effectively to unseen data.

When comparing Uni-Modal and Multi-Modal approaches, it becomes evident that incorporating multiple data modalities leads to better performance. For instance, the progression from Model 1 (Uni-modal with sync. 30 TDR dataset) to Model 3 (multi-modal with sync. 30 s11 and TDR dataset) and Model 5 (trained multi-modal with sync. 30 s11 and TDR dataset) shows a trend

of increasing accuracy. This improvement can be attributed to the complementary information provided by s11 dataset. Each modality may capture unique aspects of the data, allowing the model to form a more comprehensive understanding of the task and reducing the risk of overfitting to noise in any single modality.

However, not all Multi-Modal configurations perform equally well. The Multi-Modal model that includes missing data imputation (s11: 30 sync, 45 null & TDR 75) exhibits the lowest accuracy (54.7%). This result highlights the challenge of handling incomplete data, even with imputation techniques. The likely reason for the poor performance is that imputation may introduce noise or bias, particularly when a significant portion of the data is missing. The imputed values may not accurately reflect the true distribution, leading the model to learn from potentially misleading or inaccurate inputs.

The most significant performance gain is observed in the Multi-Modal model with transfer learning, which achieves a consistent accuracy of 83.33% across all measures. This demonstrates the power of transfer learning in leveraging knowledge from a related task or domain to enhance model performance. Transfer learning likely allows the model to start from a more informed set of weights, enabling faster convergence and better generalization, especially when training data is limited or diverse in nature.

Overall, the results indicate that while increasing data volume and incorporating multiple modalities are beneficial, the most effective improvement comes from transfer learning. This suggests that building on prior knowledge not only accelerates learning but also enhances the model's ability to capture complex patterns in multi-modal datasets.

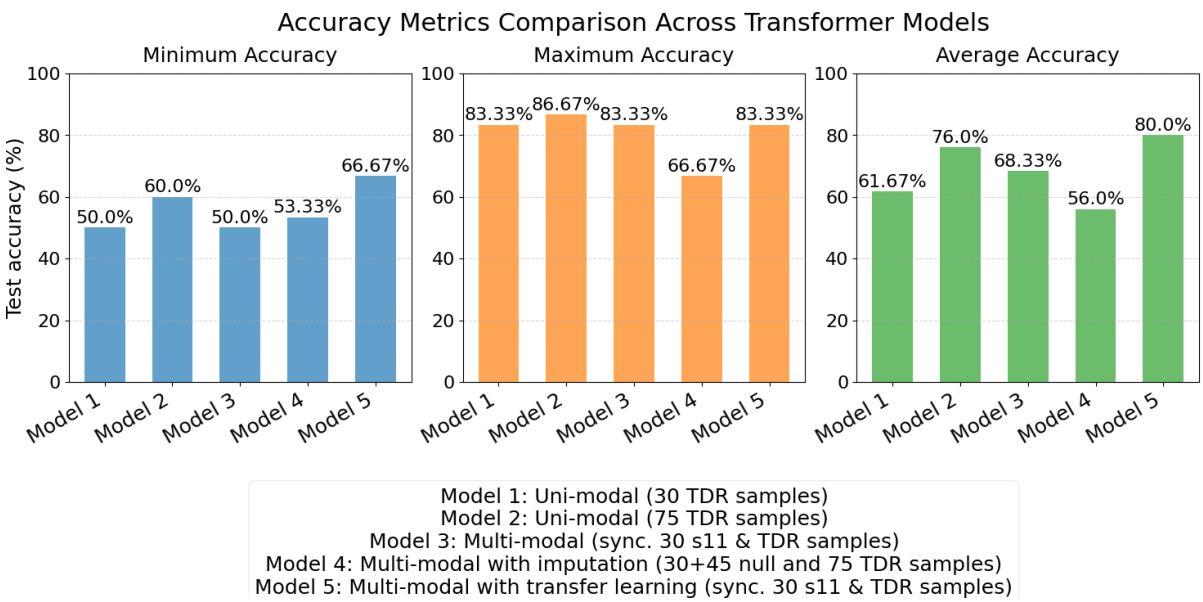


Figure 5.5: Comparative results of Transformer models

6 Conclusion and Outlook

This thesis explores to identify an effective approach for combining multi-modal data for improved failure detection in GaN HEMT transistors. In chapter 3, the suitability of various algorithms is discussed. Transformer architecture is best suited for the given application considering the S-parameter and TDR data of GaN HEMT transistor. Additionally, this thesis aims to create an automatic data pre-processing workflow, reducing manual efforts in feature selection and feature engineering. To start with the data, understanding the available data and selecting the suitable data that must be considered for the studies and can lead to better results. These functions create respective datasets from the S-parameter and TDR data. Also, it finds synchronized samples based on the value of V_{ds}, V_{gs}, L_{g} and DUT values, accordingly synchronized S-parameter and TDR dataset is created.

Initially, a Uni-modal Transformer model is implemented to understand the transformer model suitability and to analyze its efficiency for the failure classification task. The performance of the Uni-modal model is evaluated using both the synchronized TDR dataset and the entire TDR dataset. The analysis demonstrated that the model's performance improved as the dataset size increased. Furthermore, various model parameters were optimized based on performance evaluations. The influence of data scaling techniques, specifically Min-Max scaling and Standard scaling, was also evaluated. The results showed minimal variation in model performance between the two scaling methods. Therefore, for further evaluation of multi-modal transformer models, only the standard scaler technique was considered.

Building upon the initial architecture, an additional layer was integrated into the transformer model to process the S-parameter dataset. The model employed an encoder–embedding structure to extract high-level features from both the S-parameter and TDR datasets. These features were then concatenated and passed to a shared classifier, allowing the model to learn a more comprehensive representation of the input and thereby improving failure classification accuracy. The performance of this multi-modal model was evaluated on the synchronized dataset, where it outperformed the uni-modal baseline. Specifically, while the uni-modal model on the synchronized dataset achieved an accuracy of 61.67%, the initial multi-modal configuration without transfer learning improved this to 68.33%. When transfer learning was introduced, the accuracy further increased to 80.00%, demonstrating the model's enhanced ability to generalize from limited and diverse data. This 20% improvement over the uni-modal and 11.67% improvement with transfer

learning highlights the effectiveness of combining data of s-parameter and TDR and the effective use of transfer learning for improvement in the classification outcomes.

There are notable challenges and limitations are faced during this thesis. The total number of samples in the dataset is limited. This puts a limit on the usability of the data for the failure classification task. Additionally, very few samples are found overlapping on S-parameter and TDR dataset. This limited number of samples has a direct impact on the Model performance and the training phase of the model. Initially, when the model starts with a training loop, the model tends to overfit and gradually, it fits properly with the data and yields better accuracy with the training and validation set. This makes the model exhibit prolonged training time while yielding suboptimal accuracy on the test dataset. This suboptimal accuracy on the synchronized dataset has been improved with the approach of Transfer learning. Using augmented data of S-parameter to make dataset of S-parameter and TDR of equal size. Utilizing the dataset for training of the model and this trained model lead to better accuracy on the synchronized dataset.

Future improvements can focus on enhancing the transformer architecture for multi-modal classification tasks. Modifying core components of the transformer, such as attention mechanisms, positional encoding strategies and model depth, may allow for more effective representation of diverse modalities. Customizing these elements based on the characteristics of the s-parameter and TDR data can lead to more accurate and robust feature extraction. In addition, hyperparameter tuning remains a critical step for performance optimization. The careful adjustment of training parameters can significantly influence the model's ability to generalize and converge efficiently. Also, different modality imputation techniques can be adapted to improve the model performance, such that the model's performance will improve on synchronized s-parameter and TDR dataset.

Bibliography

[1] Lucas Agostinho et al. "A Practical Survey on Visual Odometry for Autonomous Driving in Challenging Scenarios and Conditions". In: *IEEE Access* 10 (Aug. 2022). DOI: 10.1109/ACCESS.2022.3188990.

[2] Tanjim Taharat Aurpa, Rifat Sadik, and Md Shoaib Ahmed. "Abusive Bangla comments detection on Facebook using transformer-based deep learning models". In: *Social Network Analysis and Mining* 12 (Dec. 2021). DOI: 10.1007/s13278-021-00852-x.

[3] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. "Neural Machine Translation by Jointly Learning to Align and Translate". In: *International Conference on Learning Representations (ICLR)*. 2015.

[4] Tadas Baltrušaitis, Chaitanya Ahuja, and Louis-Philippe Morency. "Multimodal Machine Learning: A Survey and Taxonomy". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 41.2 (2019), pp. 423–443.

[5] Math H. J. Bollen. *Understanding Power Quality Problems: Voltage Sags and Interruptions*. IEEE Press, 2000. ISBN: 978-0-7803-4713-7.

[6] Tom Brown et al. "Language models are few-shot learners". In: *Advances in Neural Information Processing Systems (NeurIPS)* 33 (2020), pp. 1877–1901.

[7] Kyunghyun Cho et al. "Learning phrase representations using RNN encoder-decoder for statistical machine translation". In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014, pp. 1724–1734.

[8] Junyoung Chung et al. "Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling". In: *Advances in Neural Information Processing Systems (NIPS) Workshop*. 2014.

[9] Jacob Devlin et al. "BERT: Pre-training of deep bidirectional transformers for language understanding". In: *arXiv preprint arXiv:1810.04805* (2018).

[10] Alexey Dosovitskiy et al. "An image is worth 16x16 words: Transformers for image recognition at scale". In: *arXiv preprint arXiv:2010.11929* (2020).

[11] Yan Gao, Fenglian Xiao, and Shengwei Wang. "A Review of Fault Detection and Diagnosis for Refrigeration Systems". In: *Energy Procedia* 152 (2018), pp. 519–524. DOI: 10.1016/j.egypro.2018.09.188.

[12] Felix A Gers, Jürgen Schmidhuber, and Fred Cummins. "Learning to forget: Continual prediction with LSTM". In: *Neural Computation* 12.10 (1999), pp. 2451–2471.

[13] Alex Graves, Abdel-rahman Mohamed, and Geoffrey Hinton. "Speech recognition with deep recurrent neural networks". In: *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*. IEEE. 2013, pp. 6645–6649.

[14] Kaiming He et al. "Deep residual learning for image recognition". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2016), pp. 770–778.

[15] Asmaul Hosna et al. "Transfer learning: a friendly introduction". In: *Journal of Big Data* 9.1 (2022), p. 102. DOI: 10.1186/s40537-022-00652-w. URL: https://journalofbigdata.springeropen.com/articles/10.1186/s40537-022-00652-w.

[16] Ruijia Huang, Dianqi Xu, and Hong Yu. "Multimodal Learning With Transformers: A Survey". In: *IEEE Xplore* (2023). URL: https://ieeexplore.ieee.org/document/10123038.

[17] H. Kambayashi et al. "Off-state stress-induced degradation in AlGaN/GaN HEMTs studied using S-parameters". In: *Japanese Journal of Applied Physics* 53.10 (2014), p. 100211. DOI: 10.7567/JJAP.53.100211.

[18] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. "Imagenet classification with deep convolutional neural networks". In: *Advances in Neural Information Processing Systems (NeurIPS)* (2012), pp. 1097–1105.

[19] Firas Laakom et al. "Convolutional autoencoder-based multimodal one-class classification". In: *arXiv preprint arXiv:2309.14090* (2023).

[20] Yann LeCun et al. "Gradient-based learning applied to document recognition". In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.

[21] Zhen Li and Steven C. H. Hoi. "Online Feature Selection in High-Dimensional Streaming Data". In: *IEEE Transactions on Knowledge and Data Engineering* 30.11 (2018), pp. 2103–2116.

[22] Xiaodan Liang, Lin Lee, and Chun-Yen Wang. "Multimodal Intelligence: Representation Learning, Information Fusion, and Applications". In: *arXiv preprint arXiv:1911.03977* (2019). URL: https://arxiv.org/pdf/1911.03977.

[23] A. Lidow et al. "Application of time domain reflectometry for failure localization in GaN-based devices". In: *Journal of Electronic Materials* 39.6 (2010), pp. 824–829. DOI: 10.1007/s11664-010-1434-7.

[24] M. Liu et al. "Deep learning-based multimodal fusion for missing modality imputation". In: *IEEE Transactions on Neural Networks and Learning Systems* (2018).

[25] Yifan Liu, Mei Chen, and Hao Zhang. "Multimodal Failure Detection in Power Semiconductor Modules Using Thermal and Electrical Data Fusion". In: *IEEE Transactions on Industrial Electronics* 68.9 (2021), pp. 8902–8911.

[26] Ilaria Manco et al. "Learning music audio representations via weak language supervision". In: (Dec. 2021). DOI: 10.48550/arXiv.2112.04214.

[27] Umesh K. Mishra, Primit Parikh, and Yi-Feng Wu. "AlGaN/GaN HEMTs—An overview of device operation and applications". In: *Proceedings of the IEEE* 90.6 (2002), pp. 1022–1031.

[28] Ned Mohan, Tore M. Undeland, and William P. Robbins. *Power Electronics: Converters, Applications, and Design*. 3rd. Wiley, 2003. ISBN: 978-0-471-22693-2.

[29] Jiquan Ngiam et al. "Multimodal Deep Learning". In: *Proceedings of the 28th International Conference on Machine Learning (ICML)*. ACM, 2011, pp. 689–696.

[30] Carlos Olivares et al. "Predicting Power Electronics Device Reliability under Extreme Conditions with Machine Learning Algorithms". In: *arXiv preprint arXiv:2107.10292* (2021).

[31] Sinno Jialin Pan and Qiang Yang. "A Survey on Transfer Learning". In: *IEEE Transactions on Knowledge and Data Engineering* 22 (Oct. 2010), pp. 1345–1359. DOI: 10.1109/tkde.2009.191.

[32] Majdi I. Radaideh et al. "Early Fault Detection in Particle Accelerator Power Electronics Using Ensemble Learning". In: *International Journal of Prognostics and Health Management* 14.1 (2023), pp. 1–10. URL: https://papers.phmsociety.org/index.php/ijphm/article/view/3419.

[33] Majdi I. Radaideh et al. "Fault Prognosis in Particle Accelerator Power Electronics Using Ensemble Learning". In: *arXiv preprint arXiv:2209.15570* (2022).

[34] Dipanjan Sarkar. *Practical Machine Learning with Python*. Apress, 2019.

[35] Yong Shi et al. "Methodologies for Cross-Domain Data Fusion: An Overview". In: *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 45.3 (2015), pp. 319–332. DOI: 10.1109/TSMC.2014.2358639. URL: https://ieeexplore.ieee.org/document/7230259.

[36] Karen Simonyan and Andrew Zisserman. "Very deep convolutional networks for large-scale image recognition". In: *arXiv preprint arXiv:1409.1556* (2014).

[37] William Sleeman, Rishabh Kapoor, and Preetam Ghosh. "Multimodal Classification: Current Landscape, Taxonomy and Future Directions". In: *IEEE Access* (Sept. 2021). DOI: 10.48550/arXiv.2109.09020.

[38] Zhihong Tian et al. "Deep Learning and Dempster-Shafer Theory Based Insider Threat Detection". In: *Mobile Networks and Applications* (Oct. 2020). DOI: 10.1007/s11036-020-01656-7.

[39] Matteo Tiezzi et al. "State-Space Modeling in Long Sequence Processing: A Survey on Recurrence in the Transformer Era". In: *arXiv preprint arXiv:2406.09062* (2024). URL: https://arxiv.org/abs/2406.09062.

[40] Yao-Hung Hubert Tsai, Shaojie Bai, and Louis-Philippe Morency. "Multimodal Co-learning: Challenges, Applications with Datasets, Recent Advances, and Future Directions". In: *arXiv preprint arXiv:2107.13782* (2019). URL: https://arxiv.org/pdf/2107.13782.

[41] Ashish Vaswani et al. "Attention is all you need". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017, pp. 5998–6008.

[42] Dong Wang et al. "Automatic Modulation Classification Based on CNN-Transformer Graph Neural Network". In: *Sensors* 23.16 (2023), p. 7281. DOI: 10.3390/s23167281. URL: https://www.mdpi.com/1424-8220/23/16/7281.

[43] Lijuan Wang, Rajeev Kumar, and Sheng Li. "Enhanced Parameter Estimation in Photovoltaic Cells via Fusion of Photoluminescence and Electrical Measurements". In: *Solar Energy Materials and Solar Cells* 234 (2022), p. 111374.

[44] Wei-Yao Wang, Du Tran, and Matt Feiszli. "What Makes Training Multi-Modal Classification Networks Hard?" In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020.

[45] Karl Weiss, Taghi M. Khoshgoftaar, and DingDing Wang. "A survey of transfer learning". In: *Journal of Big Data* 3 (May 2016). DOI: 10.1186/s40537-016-0043-6.

[46] Kun Xia et al. "Binary classification model based on machine learning algorithm for the DC serial arc detection in electric vehicle battery system". In: *IET Power Electronics* 12.1 (2019), pp. 112–119. DOI: 10.1049/iet-pel.2018.5789.

[47] Chang Xu, Dacheng Tao, and Chao Xu. "A Survey on Multi-view Learning". In: *Neurocomputing* 215 (2013), pp. 227–237.

[48] Shuping Yuan et al. "Cross-modal multi-label image classification modeling and recognition based on nonlinear". In: *Nonlinear Engineering* 12.1 (2023), p. 202200194.

[49] M. Zegaoui et al. "Highlighting trapping phenomena in microwave GaN HEMTs by low-frequency S-parameters". In: *International Journal of Microwave and Wireless Technologies* 5.2 (2013), pp. 137–145. DOI: 10.1017/S175907871200084X.

[50] Peng Zhang et al. "Forecasting Power Transformer Reliability Based on Thermal Loading". In: *IEEE Transactions on Power Delivery* 31.2 (2016), pp. 395–403. DOI: 10.1109/TPWRD.2015.2441042.

[51] Y. Zhang et al. "Short-Circuit Failure Modes and Mechanism Investigation of Ohmic-Gate GaN HEMT". In: *IEEE Transactions on Electron Devices* 69.10 (2022), pp. 6135–6142. DOI: 10.1109/TED.2022.3200781. URL: https://ieeexplore.ieee.org/document/10271247.

[52] Fuzhen Zhuang et al. "A Comprehensive Survey on Transfer Learning". In: *arXiv:1911.02685 [cs, stat]* (Dec. 2019). URL: https://arxiv.org/abs/1911.02685.